AI_103_AZ_Eng_Associate

All Products Change w/o Notice & Why these notes seem loose. TD

Contents

General Notes 3
Copilot Overview: Exam Focus 3
"Mechanic's Quick Reference Grid" 4
Fast Student Memory Aid 5
Quick Decision Tree 6
Foundry Parts: The "Mechanic's View" 7
Portal & VS Code Project Screenshot 8
RBAC "How Far Can They Go?" 9
Agent Security Boundary & Agent 365 9
Billing Units (What Actually Gets Charged) 10
Governance Boundary Cheat Sheet 11
Quick Mental Model 12
General Screenshots Update UI 18
Indent Errors Python 20
It's Just Text: Code, Script, JSON, ARM, MCP 20
Check what is Installed 22
Python environments 22
Skillable Lab List 24
Git Hubs not 1 repo 25
Very Loose Lab Notes to Erase Create Project 30
Prepare for an AI development project test cloud slice 31
Error part 2 – Be sure Project is named & Correct Region 34
Things 2 Check w Tim Screenshots 36
Browse Foundry Portal & Tools 41
VS Code – Foundry Extension – Choose Project 43
Command Palette 44
Open/Clone Git Repo 45

New Foundry vs Classic 45
log shows for Deny Create error: Incorrect Project Name/Region 46
Lab Test Cloud Slice/Select New Foundry vs Classic 49
Resolving "Resource creation blocked by policy AI-3016:Lab00 (KW)" in Azure 57
The Big Leap Into Foundry 59
Foundry mental model 59
The 80% Repeated Pattern 59
FALSE = Labs That Are Actually Different 62
Lab 2 - Model Evaluation 62
Lab 5 - Guardrails 63
Lab 8 - MCP 63
Lab 9 - Foundry IQ 64
Labs 10 & 11 - Agent Framework 64
Labs 12-17 65
Labs 18-22 65
What I Would Memorize for the Exam 66
Separate recurring costs from one-off services 67
Recurring Consumption Costs (run every time users interact) 67
Mostly One-Off Setup Costs 69
Labs Grouped by Cost Profile 71
Mostly Setup / Configuration 71
Mainly Operational Consumption 71
Exam Shortcut 72
Tool Calling vs MCP 74
Runtime Consumption 75
Input Tokens 75
Output Tokens 76
Manage Tokens: Set Amount 77
Context Window 77
Text Context 78
Image Context 78

Audio Context 78
What Is Inference? 79
What Usually Causes Recurring Charges? 79
My Mental Model 81
Provision Once 81
Pay Repeatedly 81
Pre-AI Guardrail Layer 87
References 92

General Notes

[AI-103T00-A github - GoCourse AI-103T00-A: Develop AI apps and agents on Azure - Training | Microsoft Learnogle Search]

What do You Want to Be when You Grow Up?

Copilot prompt: What is the tool/app the end user will use aka chat etc.. The Intent of agent to do what? Notify who Human Approve how Guardrails done by Billing models or units used
In a neat grid for a student to (almost) quick reference somewhat like a mechanic working on a car vs a sales pitch. see attached doc for lab steps to use as summary focus.

(Summarizing documents, processing invoices, managing support tickets, or publishing blog posts. In all cases, the goal is the same: freeing people and resources to focus on higher-value work by offloading repetitive and predictable tasks.

Large language models (LLMs) introduce a new type of automation with systems that can understand unstructured data, make decisions, and generate content.)

Copilot Overview: Exam Focus

Based on your notes, here's a **"Mechanic's Quick Reference Grid"**.

The focus is not marketing language, but answering the practical questions a student typically asks while working through the labs. Information is generalized from the lab objectives in AI Prompts EMPTYWHNTHRIU.docx.

"Mechanic's Quick Reference Grid"

Phase / Lab Area	End User Tool / App	Agent Intent (What is it trying to do?)	Notify Who?	Human Approval?	Guardrails Managed By	Billing Unit / Meter
Core Chat App (Labs 3-4)	Web Chat, Teams App, Custom UI	Answer questions, generate content, use company knowledge via RAG	End User	Usually No	Azure AI Content Safety, Prompt Design	Input Tokens + Output Tokens + Search Calls
RAG App (Lab 4)	Chat Interface	Retrieve enterprise documents before answering	End User	Usually No	Search Index ACLs + Content Filters	Tokens + Azure AI Search Queries
Content Filtering (Lab 5)	Any AI Front End	Block harmful prompts or responses	Security/Admin Team	Policy Driven	Azure AI Content Safety	Safety Evaluations + Tokens
Basic AI Agent (Lab 6)	Foundry Portal, Copilot-like UI	Execute assigned tasks using built-in tools	User Requestor	Optional	Agent Instructions + Content Filters	Tokens + Tool Usage
Custom Function Agent (Lab 7)	Chat/Agent UI	Call specific business functions or APIs	User Requestor	Often Before Action	Function Permissions	Tokens + API Consumption
MCP Agent (Lab 8)	Agent UI	Access external tools through MCP standard	User Requestor	Usually Configurable	MCP Permissions + Agent Policies	Tokens + Tool Calls
Foundry IQ Integration (Lab 9)	Agent Dashboard	Analyze operational and enterprise intelligence	Business Users	Usually No	Data Governance Controls	Tokens + Foundry Services

Phase / Lab Area	End User Tool / App	Agent Intent (What is it trying to do?)	Notify Who?	Human Approval?	Guardrails Managed By	Billing Unit / Meter
Agent Framework SDK (Lab 10)	Custom Application	Programmatic orchestration and automation	App User	Code Defined	Developer Controls	Tokens + Compute
Multi-Agent System (Lab 11)	Shared Agent Workspace	Delegate work among specialist agents	User Requestor	Optional Handoff Approval	Agent Roles & Policies	Tokens Across All Agents
Text Analysis Agent (Labs 12-13)	Analysis App	Extract entities, sentiment, keywords	Analyst	Usually No	Content Filters	Requests or Tokens
Speech Agent (Labs 14-17)	Voice Chat, Call Center UI	Listen, understand, respond with audio	Caller/User	Usually No	Speech Moderation + Safety Filters	Audio Minutes + Tokens
Vision Chat (Lab 18)	Multimodal Chat	Analyze images and answer questions	User	Usually No	Image Safety Policies	Image Tokens + Text Tokens
Image Generation (Lab 19)	Image Studio, App UI	Create images from prompts	User	Usually No	Content Safety for Images	Images Generated
Content Understanding (Labs 20-22)	Document Processing App	Extract fields, tables, OCR content	Business User	Usually No	Data Classification & Security	Pages Processed / Transactions

Fast Student Memory Aid

If You Are Building...	Think Like...
Chat App	AI answers questions
RAG App	AI reads manuals before answering

If You Are Building...	Think Like...
Agent	AI can decide to use tools
Function Calling	AI presses buttons you expose
MCP	AI plugs into external systems
Multi-Agent	Team of specialists working together
Speech Agent	AI call center worker
Vision App	AI can see images
Image Generator	AI artist
Content Understanding	AI document clerk extracting data

Quick Decision Tree

Need answers only? → Chat App

Need company documents? → RAG

Need actions performed? → Agent

Need external systems/tools? → MCP

Need approval before action? → Agent + Human-in-the-Loop

Need voice? → Speech Agent

Need images analyzed? → Vision

Need images created? → Image Generation

Need forms, invoices, receipts processed? → Content Understanding

This is essentially the**"what wrench do I grab for this repair?"** version of the AI-103 lab sequence instead of the Microsoft marketing view. [AI Prompts...TYWHNTHRIU | Word]

Boundaries, Billing, Governance Nutshell

For someone coming from the traditional Azure hierarchy mindset, I think of **Azure AI Foundry** like this:

Management Group (optional)

|

```
└──── Subscription
  │
  └──── Resource Group
    │
    └──── Foundry Resource (Account)
      │
      ├──── Project A
      │     ├──── Agents
      │     ├──── Models
      │     ├──── Connections
      │     └──── Evaluations
      │
      └──── Project B
          ├──── Agents
          ├──── Models
          └──── Assets
```

The key insight is that **Project is the primary security and development boundary**, while the **Foundry Resource/Account is the governance boundary**. [learn.microsoft.com], [github.com]

Foundry Parts: The "Mechanic's View"

Layer	Similar Azure Concept	Purpose	RBAC Boundary
Subscription	Subscription	Billing owner	Azure RBAC
Resource Group	Resource Group	Resource container	Azure RBAC
Foundry Resource (Account)	Azure OpenAI Resource / ML Workspace	Governance, quotas, models, projects	Foundry Roles
Project	Dev Workspace	Team builds apps & agents	Foundry Roles

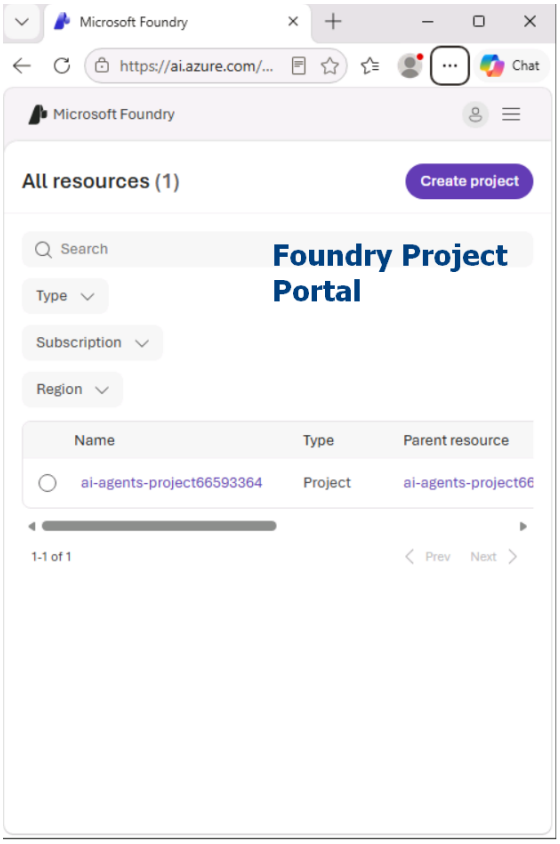
Layer	Similar Azure Concept	Purpose	RBAC Boundary
Agent	Application Object	Runtime behavior	Inherits Project permissions
Agent Identity	Managed Identity	Accesses downstream resources	Azure RBAC

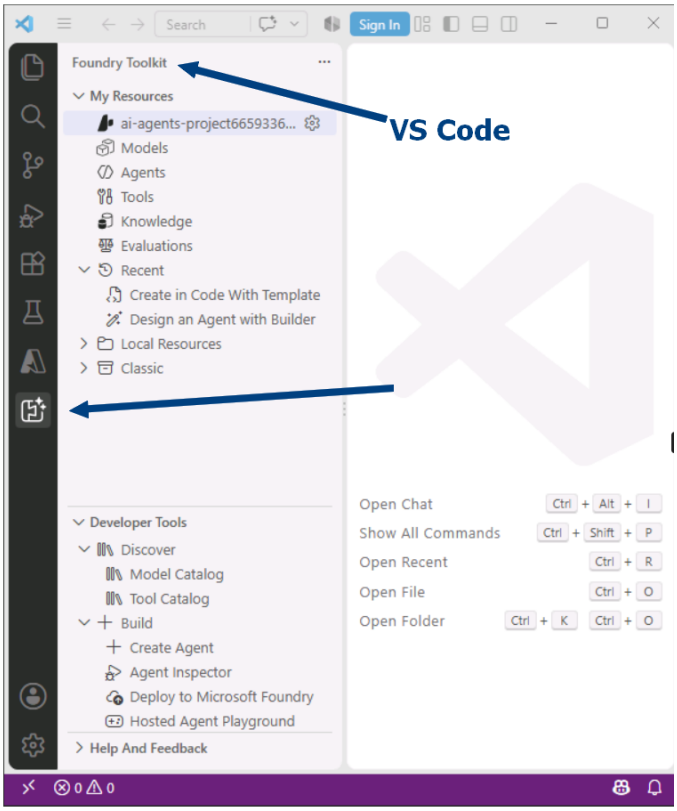
Microsoft explicitly treats the Foundry resource as the parent scope and Projects as contained workspaces beneath it. [learn.microsoft.com], [learn.microsoft.com]

Portal & VS Code Project Screenshot

The resource level relates to the Foundry resource that was created in Azure to support your project. This resource includes connections to Foundry Services and models; and provides a central place to manage user access to AI development projects.

The project level relates to your individual project, where you can add and manage project-specific resources. A resource can support multiple projects (the first one created is the resource's default project).





RBAC "How Far Can They Go?"

Role	Create Projects	Build Agents	Publish Agents	Assign Roles
Foundry User	No	Yes	No	No
Foundry Project Manager	Yes	Yes	Yes	Limited
Foundry Account Owner	Yes	Yes	Yes	Yes
Foundry Owner	Yes	Yes	Yes	Yes
Azure Owner	Everything	Everything	Everything	Everything

These are Foundry-specific roles layered on top of normal Azure RBAC. [learn.microsoft.com], [github.com]

Agent Security Boundary & Agent 365

An agent itself does **not** magically get access to resources.

Think:

User

↓

Agent

↓

Agent Identity

↓

Storage / SQL / SharePoint / Search

The Agent Identity (Entra ID object) must be granted permissions via Azure RBAC or the target service's permission system. Least privilege is recommended. [learn.microsoft.com], [roykim.ca]

Example:

Agent

↓

Read Blob Storage

Agent Identity needs:

Storage Blob Data Reader

on the Storage Account.

No role = No access. [learn.microsoft.com], [roykim.ca]

Billing Units (What Actually Gets Charged)

Foundry itself is mostly an orchestrator.

You usually pay for the things underneath it:

Service	Billing Unit
GPT / LLM Models	Input Tokens + Output Tokens

Service	Billing Unit
Provisioned Models	PTUs (Provisioned Throughput Units)
Azure AI Search / Foundry IQ	Search Units, Partitions, Replicas
File Search	Vector Storage GB/day
Code Interpreter	Session
Web Search	Per 1,000 Transactions
Custom Search	Per 1,000 Transactions
Speech	Audio Minutes
Vision	Images / Tokens
Hosted Agents	vCPU-hour + Memory GiB-hour

Foundry-native prompt agents themselves generally do not have an additional "agent fee"; costs come from model usage and connected services. [azure.microsoft.com], [azure.microsoft.com], [cdn-dynmed...rosoft.com]

Governance Boundary Cheat Sheet

Want To Control...	Set It Here
Cost Center	Subscription / Resource Group
Budget Alerts	Subscription
Quotas	Foundry Resource
Models Available	Foundry Resource
Who Builds Agents	Project
Who Runs Agents	Project
Agent Access To Storage	Agent Identity + Azure RBAC
Agent Access To APIs	Connection Permissions

Want To Control...	Set It Here
Network Isolation	Foundry Resource / Hub Configuration
Content Safety	Foundry Policies + Azure AI Content Safety

[learn.microsoft.com], [learn.microsoft.com], [azure.microsoft.com]

Quick Mental Model

If Azure is:

Tenant

→ Subscription

→ Resource Group

→ Resource

then Foundry is roughly:

Foundry Resource

→ Project

→ Agent

→ Agent Identity

→ External Resources

Where:

- **Billing ownership** stays mostly at Subscription/Resource Group.
- **Governance** happens at the Foundry Resource.
- **Development collaboration** happens at the Project.
- **Runtime permissions** happen through Agent Identity + Azure RBAC.

That's the closest "Azure infrastructure mechanic" mapping I've found for explaining Foundry to students and admins.

For **exam + lab purposes**, I'd separate three things:

1. **What Microsoft teaches today (new Foundry)**

2. **What the labs were originally built around (many were classic/hub-based)**
3. **What you'll actually encounter in training subscriptions and cloud slices**

New Foundry vs Classic Foundry

Classic (Hub / Project Model)

Think:

Hub

├─ Shared Networking

├─ Shared Key Vault

├─ Shared Storage

├─ Shared AI Services

└─ Projects

The Hub owned the infrastructure and Projects inherited from the Hub. This is the model used by many older AI Studio / AI-102 / early Agent labs. <Microsoft Foundry (classic)> documentation explicitly describes the Hub as the infrastructure owner and Projects as developer workspaces. [learn.microsoft.com]

Typical student experience:

- Create Hub
- Create Project under Hub
- Build agents
- Use inherited resources

New Foundry

Think:

Foundry Resource

└─ Project

├─ Models

├─ Agents

├─ Connections

└─ Evaluations

Microsoft's newer documentation focuses on a **Foundry resource/account with Projects beneath it**, rather than the older Hub-centric architecture. RBAC, projects, quotas, and agent publishing are documented around this new model. [learn.microsoft.com], [github.com]

Typical student experience:

- Create Foundry resource
- Create Project
- Deploy models
- Create agents
- Publish

Exam Perspective

If I were coaching an AI-103 student:

Know Well	Know Exists
Foundry Projects	Classic Hubs
Agent Service	AI Studio legacy terminology
Agent Identity	Hub inheritance model
RBAC roles	Classic project roles
Foundry IQ	Older AI Studio workflows

The direction of travel is clearly toward **Foundry Resource + Project + Agent** rather than Hub-centric design. [learn.microsoft.com], [learn.microsoft.com]

Why Classic Sometimes Works When New Foundry Doesn't

Your policy example is actually a good clue:

Policy: AI-3016:Lab00 (KW)

Effect: deny

Resource:

Microsoft.Insights/components

App Insights

Notice the failure isn't:

Microsoft.AI

Microsoft.CognitiveServices

Foundry

Agent

It's:

Microsoft.Insights/components

which is **Application Insights**. Application Insights is commonly created as a supporting resource during project creation and monitoring setup. The resource provider is Microsoft.Insights/components. The new Foundry experience can automatically provision monitoring resources as part of project creation workflows. [learn.microsoft.com], [learn.microsoft.com]

So from a lab admin point of view:

New Foundry

→ Tries to create Project

→ Creates Application Insights

→ Policy blocks

while:

Classic

→ Uses existing shared infra

→ No new App Insights creation

→ Succeeds

That is a plausible explanation, though your note is correct: this looks much more like a **training subscription / cloud slice policy issue** than anything architecturally wrong with Foundry.

What I'd Tell Students

If You See

Policy Effect = Deny

during Project creation:

Think:

"Azure Policy blocked an infrastructure resource."

not:

"The agent failed."

Often the failure is an underlying resource:

- Key Vault
- Storage Account
- Application Insights
- Virtual Network
- Managed Identity

rather than the Foundry project itself.

Exam Shortcut

When you see:

Microsoft.Insights/components

translate mentally to:

Application Insights

Monitoring

Telemetry

Observability

not AI functionality.

For exam questions:

Agent = Business Logic

Model = Intelligence

Project = Workspace

Application Insights = Monitoring

My "Mechanic's Cheat Sheet" for AI-103 Labs

Tenant

└ Subscription

└ Resource Group

└ Foundry Resource

└ Project

└ Agent

Supporting Resources:

Storage

Key Vault

App Insights

Search

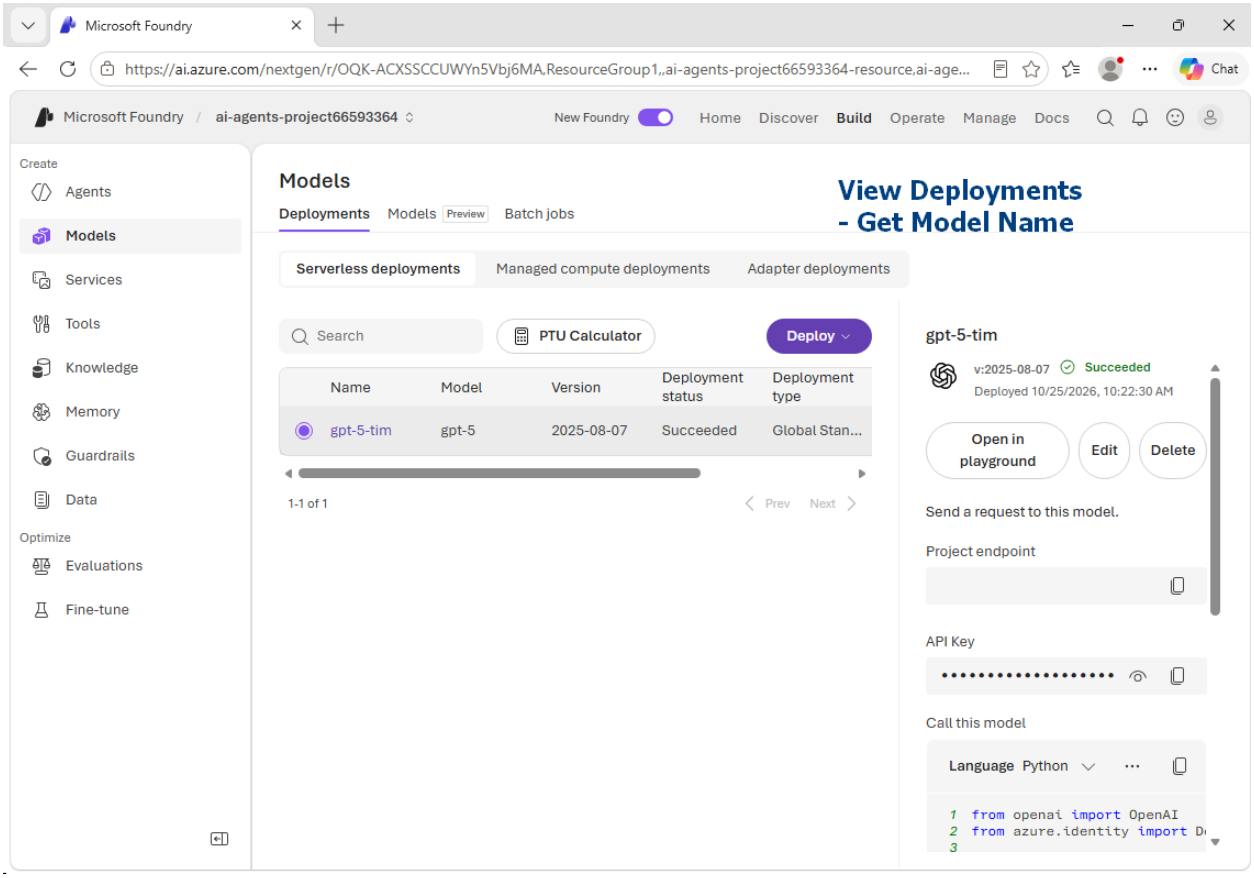
If Project Creation Fails:

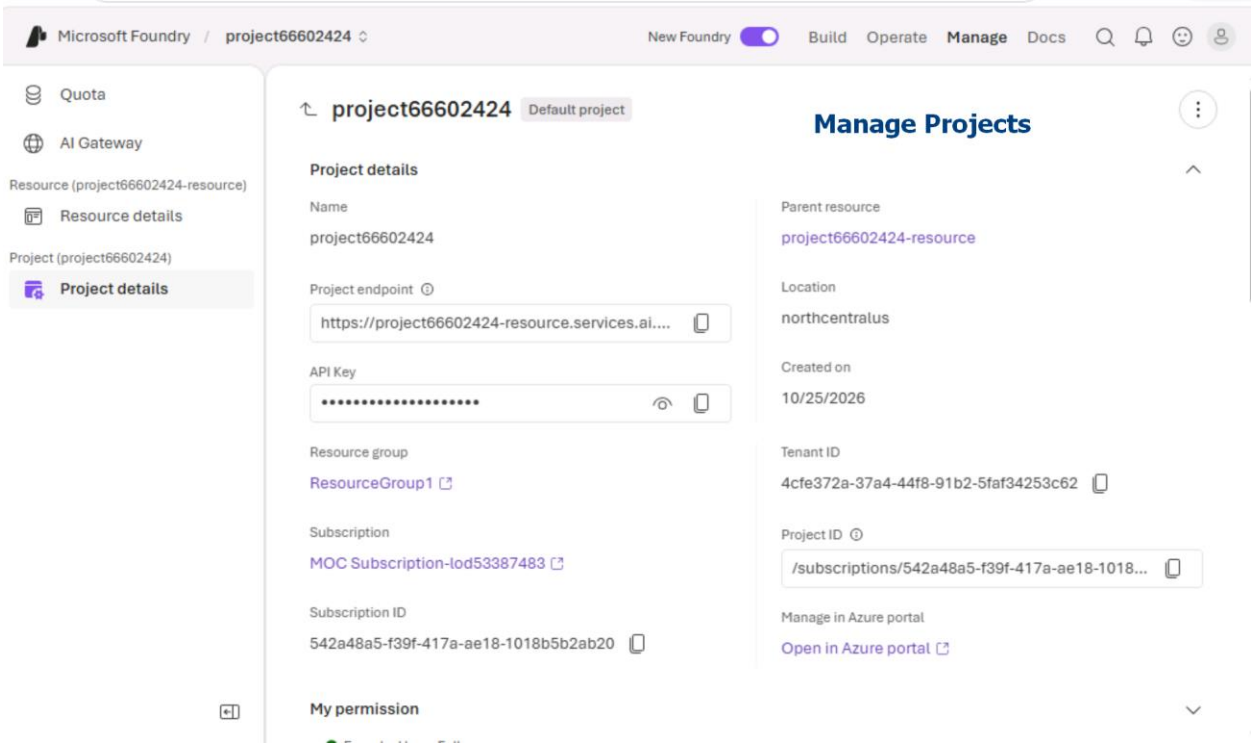
Look below the project.

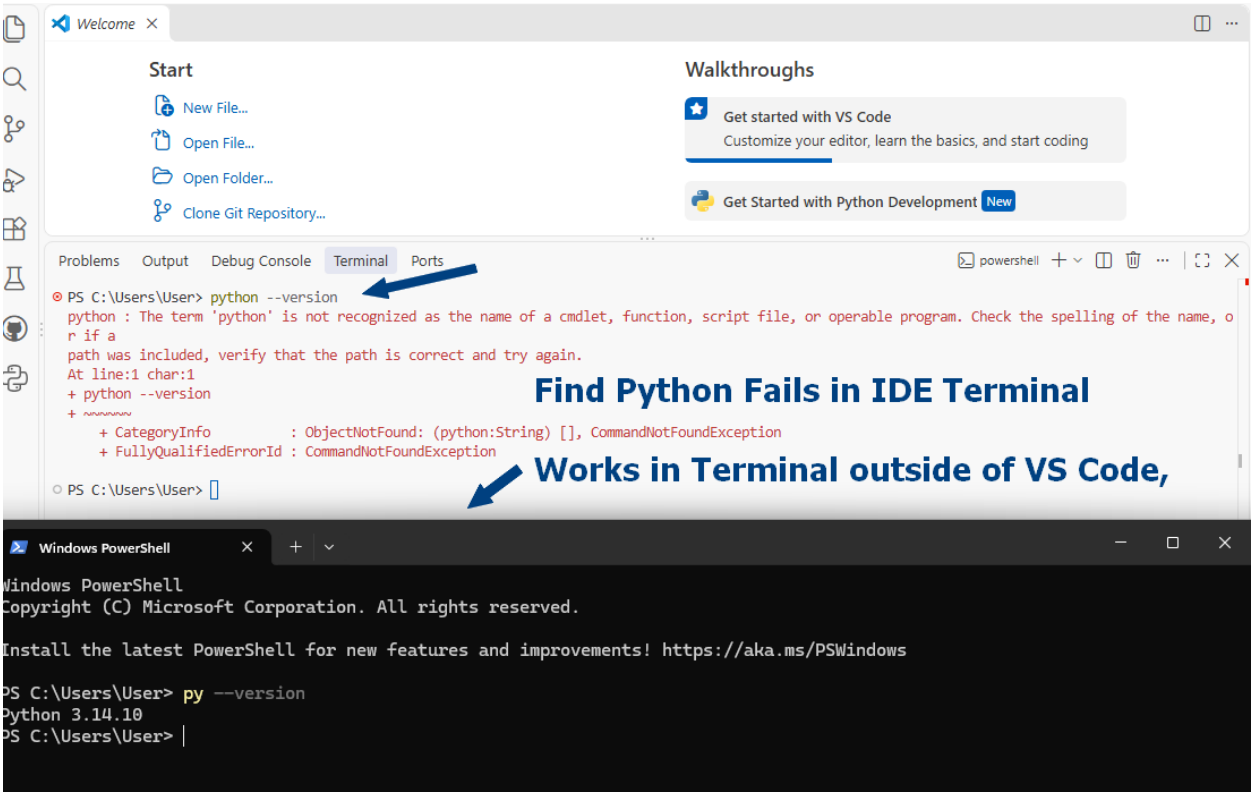
The root cause is frequently one of the supporting Azure resources being blocked by policy, quota, region restrictions, or lab subscription constraints, which aligns closely with the Microsoft.Insights/components denial you observed. [learn.microsoft.com], [learn.microsoft.com]

For AI-103 study purposes, I'd treat this as a **lab environment artifact**, not evidence that New Foundry is fundamentally replacing Application Insights or requiring radically different architecture. The security model, project model, and monitoring dependencies simply surface differently between the newer and older experiences. [learn.microsoft.com], [learn.microsoft.com]

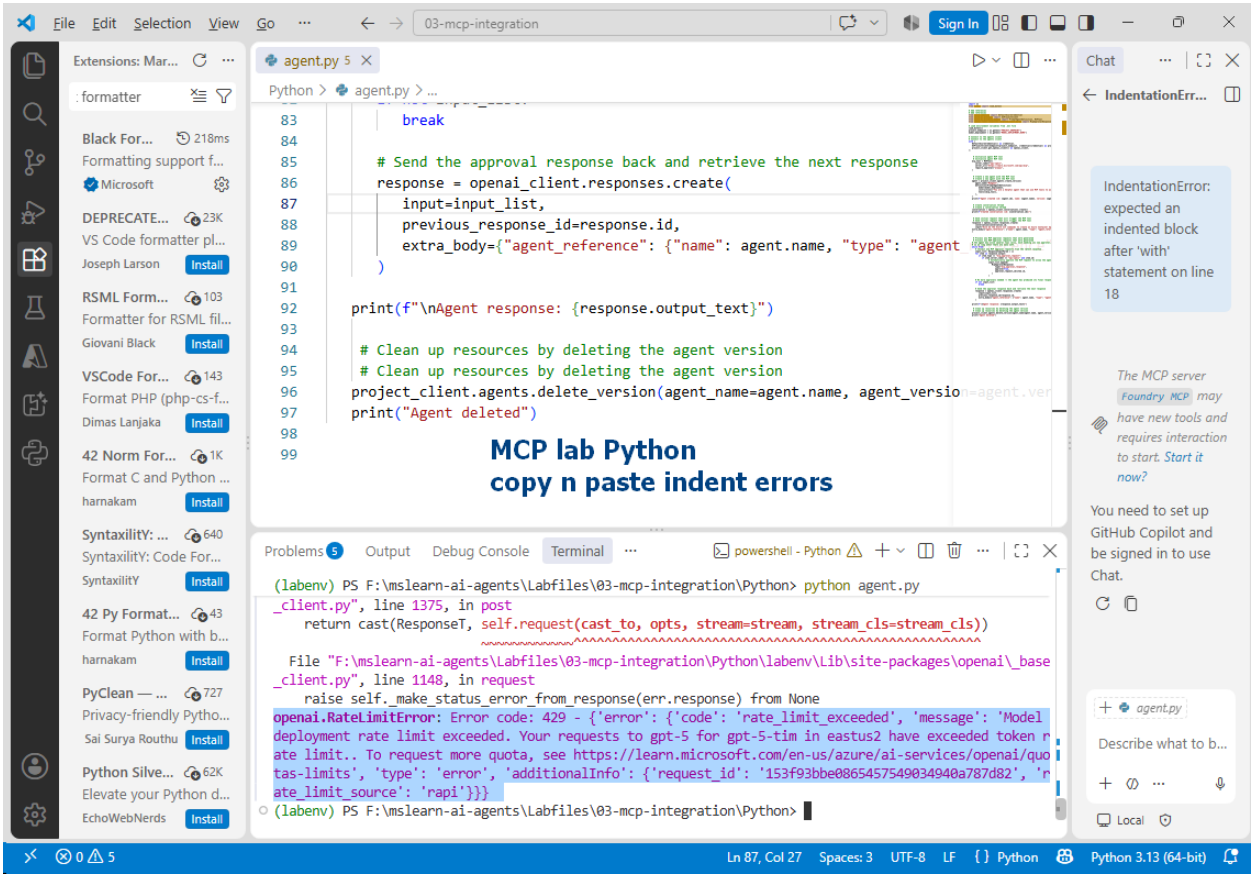
General Screenshots Update UI






Welcome
Start
New File...
Open File...
Open Folder...
Clone Git Repository...
Walkthroughs
Get started with VS Code
Customize your editor, learn the basics, and start coding
Get Started with Python Development
New
Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\Users\User> python --version
python : The term 'python' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a
path was included, verify that the path is correct and try again.
At line:1 char:1
+ python --version
+ ~~~~~~
+ CategoryInfo : ObjectNotFound: (python:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
PS C:\Users\User>
Find Python Fails in IDE Terminal
Works in Terminal outside of VS Code,
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
PS C:\Users\User> py --version
Python 3.14.10
PS C:\Users\User>

Indent Errors Python



It's Just Text: Code, Script, JSON, ARM, MCP

Yes, but it depends heavily on which platform named "Foundry" you are referring to, as there are two massive, separate AI ecosystems operating with MCP (Model Context Protocol): **Microsoft Azure AI Foundry** and **Palantir Foundry**. [1, 2]

Both platforms fully integrate **JSON** and **Python** natively into their MCP server scripting and execution flows. However, **ARM Templates** (Azure Resource Manager) are exclusively native to Microsoft's ecosystem. [1, 2, 3]

A direct breakdown shows how these technologies act natively in both environments:

1. JSON (Native to Both)

- **The Underlying Protocol:** MCP is structurally an open standard built entirely on top of **JSON-RPC 2.0**. Every request from a client to an MCP server, every response, and every tool-schema discovery payload is natively parsed as a JSON object. [1, 2, 3]

- **Configuration:** In both ecosystems, local server routing, multi-server registration environments, and VS Code extension bindings are defined natively inside mcp.json or tasks.json configuration files. [1, 2]

2. Python (Native to Both)

- **Server Scripting:** Python is the most popular native language for coding custom MCP servers. Developers use Python SDKs (like mcp or FastMCP) to write the functions that turn into LLM-callable tools. [1, 2, 3, 4]
- **Execution Environment:**
    - In **Azure AI Foundry**, Python is used alongside Azure Functions or Azure Container Apps to host remote MCP servers.
    - In **Palantir Foundry**, Python is used natively within "Code Repositories" to author Python Data Transforms or build "Pro-code Agents" that leverage the Ontology MCP (OMCP). [1, 2, 3, 4, 5]

3. ARM Templates (Native ONLY to Azure AI Foundry)

- **Infrastructure as Code: ARM Templates are 100% specific to Microsoft Azure.** [1]
- If you are scripting or deploying an MCP server context in **Azure AI Foundry**, you natively use ARM templates (or Bicep/Azure Developer CLI) to provision things like isolated Azure Container App session pools with the mcpServerSettings.isMCPServerEnabled property turned on. [1, 2]
- Palantir Foundry has no native concepts or ties to ARM templates, using its own proprietary infrastructure architecture instead. [1]

Structural Comparison of Both "Foundry" Platforms

Component	Microsoft Azure AI Foundry	Palantir Foundry
JSON	Native (Protocol communication & mcp.json declarations)	Native (Standard JSON-RPC 2.0 endpoint schemas)
Python	Native (Azure Functions remote servers & AI Project Client SDK)	Native (Python Data Transforms & Pro-code Agent repositories)
ARM Templates	**Native** (Used to spin up session pools/secure infrastructure)	**Not Native** (Completely unrelated to the Palantir cloud stack)
Primary MCP Focus	Connecting Foundry Agents outward to custom tools and APIs	Exposing the Foundry Ontology layer *inward* to IDEs like VS Code / Claude Code

Which platform are you actively building on—**Azure AI Foundry** or **Palantir Foundry**? If you are trying to write or provision a specific server, let me know the architecture you're aiming for so I can provide the exact code or script snippet.

Check what is Installed

- Visual Studio Code installed on your local machine
- Python 3.13 installed
- Git installed on your local machine

Open Visual Studio Code / View Terminal to enter commands:

Tool	Command	Expected Success Output
Git	`git --version`	`git version x.x.x`
Python	`python3 --version` *or* `python --version`	`Python x.x.x`

You **cannot** install Git itself directly from the VS Code Extensions window, but you can install it using your terminal.

macOS: brew install git

Windows PowerShell: winget install --id Git.Git -e --source winget

Linux Bash: sudo apt update && sudo apt install git -y

Or: Git - Install : https://git-scm.com/install/

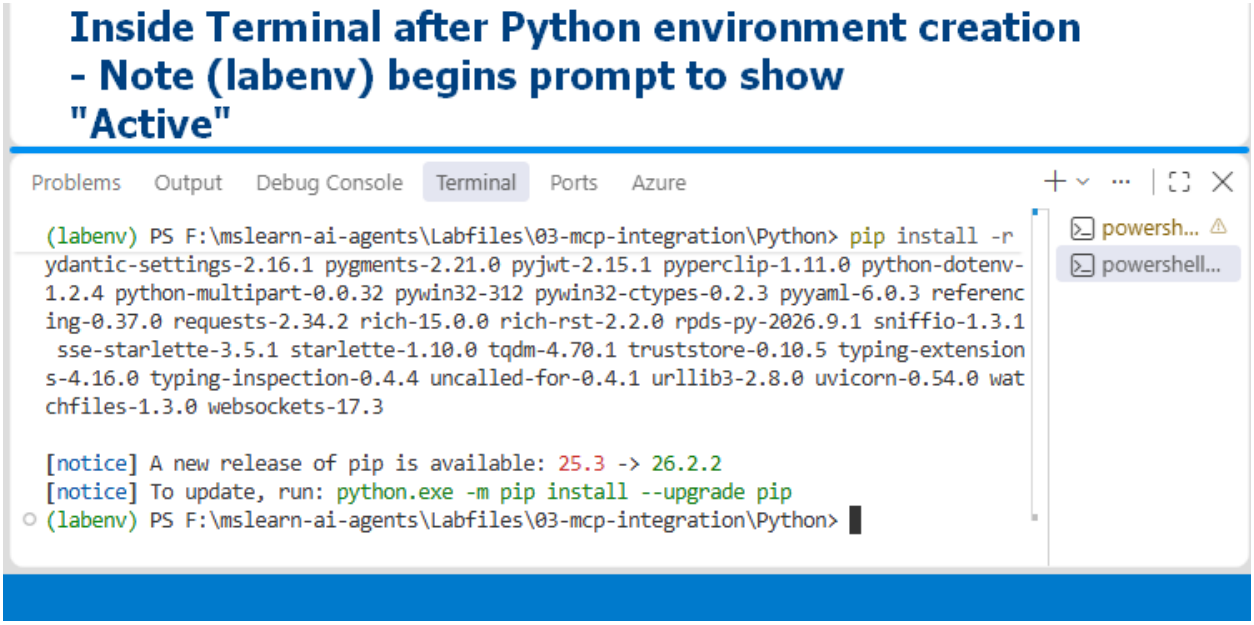
Python environments

Creates a folder path to run commands locally from text file – see screenshot for Active env.

python -m venv labenv

.\labenv\Scripts\Activate.ps1

pip install -r requirements.txt



Uvicorn is an incredibly fast, lightning-quick **ASGI (Asynchronous Server Gateway Interface) server for Python**.

Think of it as the engine under the hood that allows your modern Python web applications to run and handle thousands of requests simultaneously.

💰 The "Why" behind Uvicorn

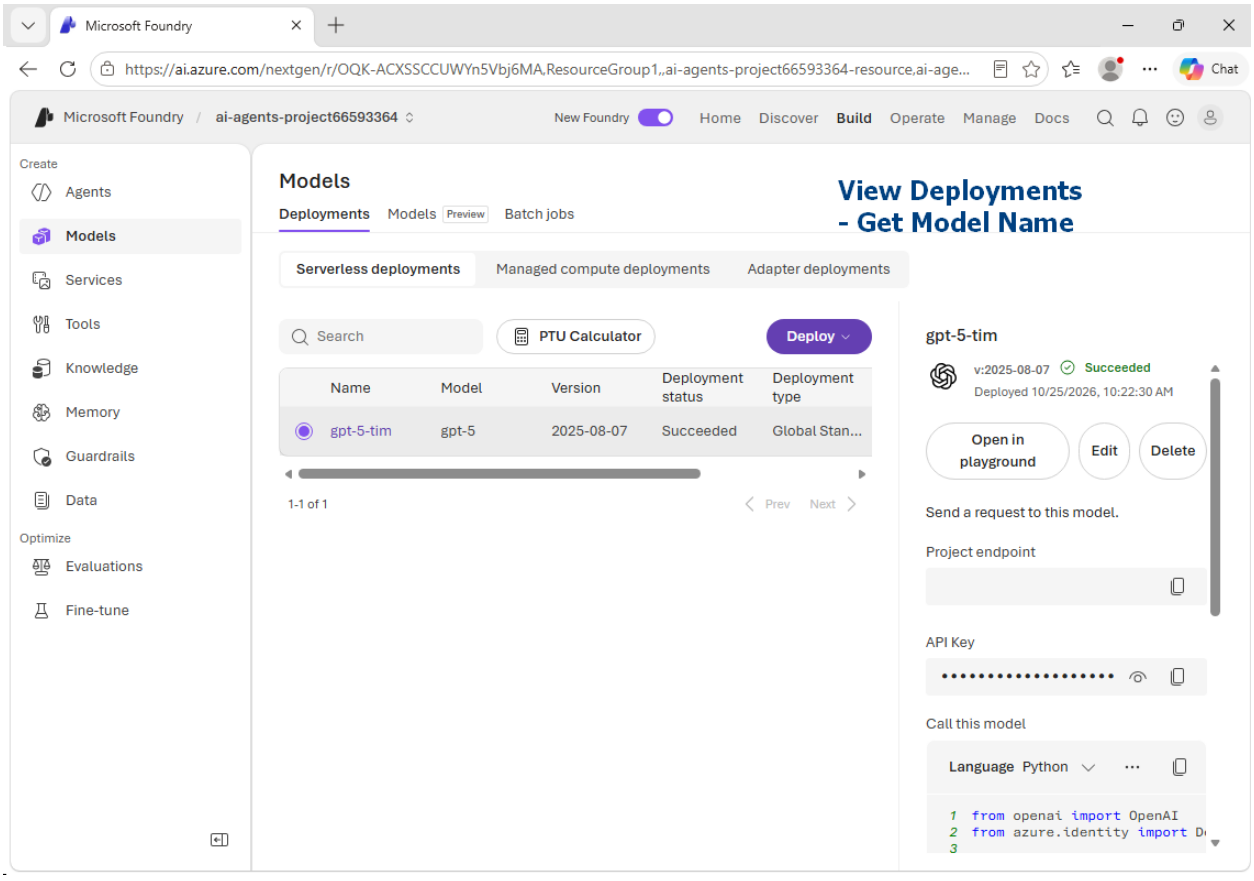
Traditional Python web frameworks (like Django or Flask) historically used **WSGI**, which handles requests sequentially (one after another). If a request takes a long time, the server waits around doing nothing.

Uvicorn uses **ASGI**, which is built for asynchronous Python (async/await). This means if one request is waiting on a database or an external API, Uvicorn immediately switches tasks to handle a different user's request.

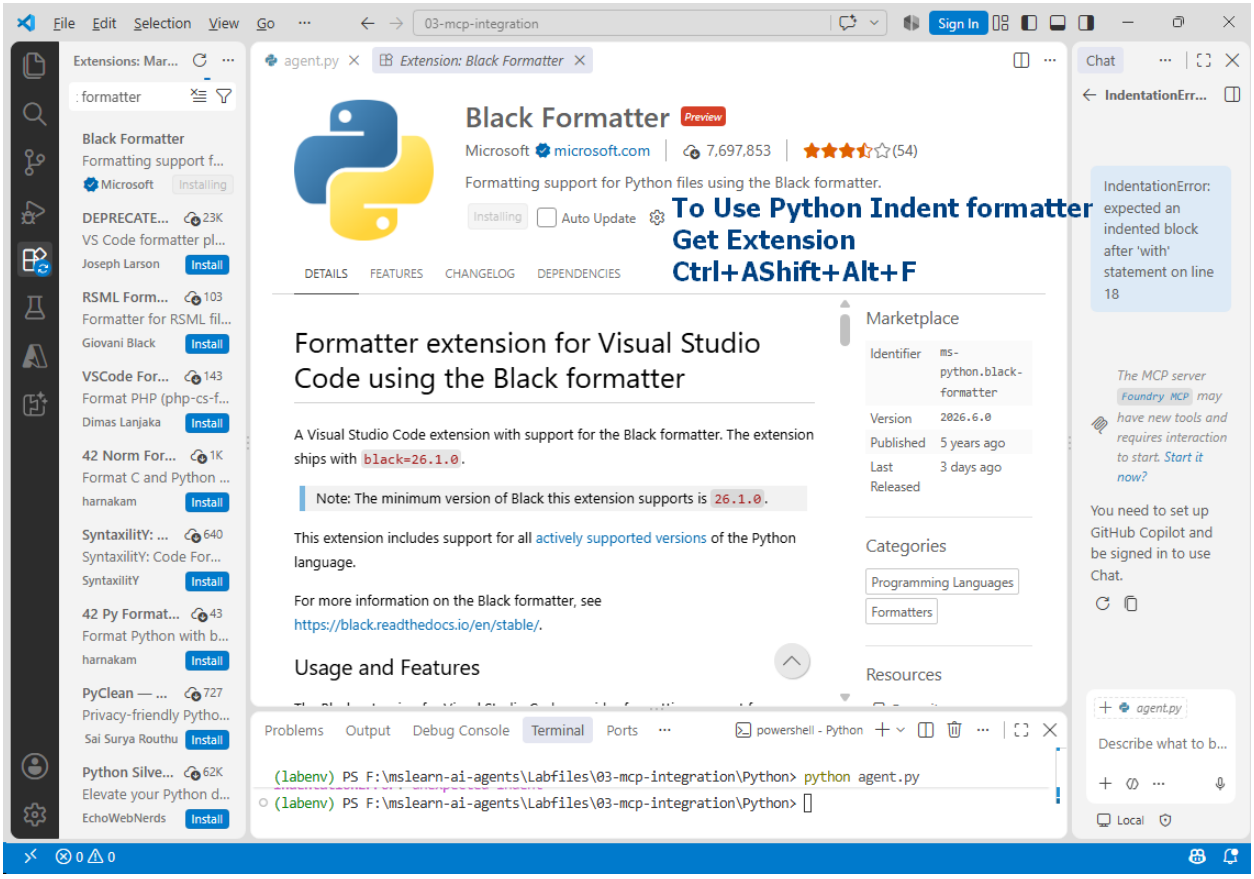
🤝 Its Best Friend: FastAPI

Uvicorn is most famous for being the standard server used to run **FastAPI**. If you are building a FastAPI app, you literally cannot preview or launch your app without Uvicorn.

View Deployment Portal – Model Name



Python Code Formatter Extension



Skillable Lab List

1. Prepare for an AI development project (1 hours)
2. Explore models and evaluate performance (1 hours, 30 minutes)
3. Create a generative AI chat app (1 hours, 30 minutes)
4. Create a generative AI app that uses your own data (2 hours, 30 minutes)
5. Apply content filters to prevent the output of harmful content (1 hours, 30 minutes)
6. Build AI agents with portal and VS Code (2 hours, 30 minutes)
7. Use a custom function in an AI agent (2 hours, 30 minutes)
8. Develop an AI agent with Model Context Protocol (MCP) tools (2 hours, 30 minutes)
9. Integrate an AI agent with Foundry IQ (2 hours, 30 minutes)
10. Develop an Azure AI chat agent with the Microsoft Agent Framework SDK (2 hours, 30 minutes)
11. Develop a multi-agent solution (2 hours, 30 minutes)
12. Analyze text (1 hours)
13. Develop A Text Analysis Agent (1 hours)
14. Use speech-capable generative AI models (1 hours)
15. Recognize and synthesize speech (1 hours)
16. Use Azure Speech in an agent (1 hours)
17. Develop a Voice Live agent (1 hours)
18. Develop a vision- chat app (1 hours, 30 minutes)
19. Generate images with AI (1 hours, 30 minutes)

20. Analyze images with Azure Content Understanding (1 hours, 30 minutes)
21. Extract information from multimodal content (Optional) (2 hours)
22. Develop a Content Understanding client application (1 hours, 30 minutes)

Text list

Prepare for an AI development project (1 hours)

Explore models and evaluate performance (1 hours, 30 minutes)

Create a generative AI chat app (1 hours, 30 minutes)

Create a generative AI app that uses your own data (2 hours, 30 minutes)

Apply content filters to prevent the output of harmful content (1 hours, 30 minutes)

Build AI agents with portal and VS Code (2 hours, 30 minutes)

Use a custom function in an AI agent (2 hours, 30 minutes)

Develop an AI agent with Model Context Protocol (MCP) tools (2 hours, 30 minutes)

Integrate an AI agent with Foundry IQ (2 hours, 30 minutes)

Develop an Azure AI chat agent with the Microsoft Agent Framework SDK (2 hours, 30 minutes)

Develop a multi-agent solution (2 hours, 30 minutes)

Analyze text (1 hours)

Develop A Text Analysis Agent (1 hours)

Use speech-capable generative AI models (1 hours)

Recognize and synthesize speech (1 hours)

Use Azure Speech in an agent (1 hours)

Develop a Voice Live agent (1 hours)

Develop a vision- chat app (1 hours, 30 minutes)

Generate images with AI (1 hours, 30 minutes)

Analyze images with Azure Content Understanding (1 hours, 30 minutes)

Extract information from multimodal content (Optional) (2 hours)

Develop a Content Understanding client application (1 hours, 30 minutes)

Git Hubs not 1 repo

Yes, the official Microsoft Learning repositories on GitHub contain the exact step-by-step instructions and code files for these labs. [1, 2]

Because Microsoft split the **AI-103: Developing AI Apps and Agents on Azure** course material across different functional domains rather than hosting it in a single repository, you will find these specific labs divided across three core official repositories: [1, 2]

---

1. 🤖 The AI Agents Repository

Labs **6 through 11**, as well as Lab **13**, are located in the MicrosoftLearning/mslearn-ai-agents repository. [1]

- **Lab 6:** Build AI agents with portal and VS Code
- **Lab 7:** Use a custom function in an AI agent
- **Lab 8:** Develop an AI agent with Model Context Protocol (MCP) tools
- **Lab 9:** Integrate an AI agent with Foundry IQ
- **Lab 10:** Develop an Azure AI chat agent with the Microsoft Agent Framework SDK
- **Lab 11:** Develop a multi-agent solution
- **Lab 13:** Develop A Text Analysis Agent [1, 2]

2. 👁 The Vision and Multimodal Repository

Labs **18 through 22** are hosted inside the MicrosoftLearning/mslearn-ai-vision repository. [1]

- **Lab 18:** Develop a vision-chat app
- **Lab 19:** Generate images with AI
- **Lab 20:** Analyze images with Azure Content Understanding
- **Lab 21:** Extract information from multimodal content
- **Lab 22:** Develop a Content Understanding client application [1]

3. 💬 The Generative AI & Language Repository

Labs **1 through 5**, as well as the text/speech labs (**12, 14, 15, 16, 17**), are housed in the core **mslearn-openai** / language repository. [1]

- **Lab 1:** Prepare for an AI development project
- **Lab 2:** Explore models and evaluate performance
- **Lab 3:** Create a generative AI chat app
- **Lab 4:** Create a generative AI app that uses your own data
- **Lab 5:** Apply content filters to prevent the output of harmful content
- **Labs 12 & 14–17:** Text analysis, speech-capable genAI, and Voice Live agent configurations. [1, 2]

💡 Quick Tip for Accessing Lab Steps

Instead of manually hunting through individual code folders, you can read the rendered click-through instructions directly on the web by altering the URL structure. For example, if you clone mslearn-ai-agents, you can read its markdown guides directly at microsoftlearning.github.io/mslearn-ai-agents. [1, 2]

Are you planning to **clone these to your local environment** via VS Code, or would you like me to walk you through setting up the **Azure AI Foundry infrastructure** requirements needed to run them?

To facilitate high-level architectural and design discussions outside the Skillable environment, this structured **linked list** maps every lab in the **AI-103T00-A** curriculum directly to its official Microsoft Learning GitHub instructions web interface.

Students can use these live web links to preview the **technical workflows**, explore the **starter code file structures**, and analyze the **implementation logic** before starting their lab timers.

🗺️ AI-103T00-A Phase 1: Core Generative AI Apps

Focus: Project instantiation, the Microsoft Foundry Responses API, and safety guardrails.

- ➡️ **Lab 1: Prepare for an AI development project**
  - *Design Focus:* Basic provisioning of the Microsoft Foundry Hub/Project architecture and local workspace initialization.
- ➡️ **Lab 2: Explore models and evaluate performance**
  - *Design Focus:* Browsing the foundational Model Catalog, comparing performance metrics, and assessing model variants.
- ➡️ **Lab 3: Create a generative AI chat app**
  - *Design Focus:* Writing the initial orchestration layer utilizing the standard OpenAI-compatible SDK and Microsoft Foundry endpoints.
- ➡️ **Lab 4: Create a generative AI app that uses your own data**
  - *Design Focus:* Implementing a custom **Retrieval-Augmented Generation (RAG)** pipeline integrated with an Azure AI Search indexing target.
- ➡️ **Lab 5: Apply content filters to prevent the output of harmful content**
  - *Design Focus:* Building custom content filter safety policies to regulate input/output toxicity parameters.

AI-103T00-A Phase 2: Agentic Architecture & Frameworks

Focus: Autonomous agent creation, custom tooling extensions, and multi-agent coordination.

- **Lab 6: Build AI agents with portal and VS Code**
  - *Design Focus:* Creating stateful cloud agents using standard native tools (e.g., Code Interpreter and File Search). [1]
- **Lab 7: Use a custom function in an AI agent**
  - *Design Focus:* Defining and exposing client-side local execution schemas that the remote agent can call dynamically.
- **Lab 8: Develop an AI agent with Model Context Protocol (MCP) tools**
  - *Design Focus:* Hooking open-standard external tools directly into the agent using the standardized **Model Context Protocol (MCP)**. [1]
- **Lab 9: Integrate an AI agent with Foundry IQ**
  - *Design Focus:* Connecting an agentic execution pipeline to enterprise telemetry and intelligence layers inside Foundry IQ.
- **Lab 10: Develop an Azure AI chat agent with the Microsoft Agent Framework SDK**
  - *Design Focus:* Moving beyond basic playground definitions to program code-first agents directly with the Microsoft Agent Framework SDK.
- **Lab 11: Develop a multi-agent solution**
  - *Design Focus:* Establishing group messaging topologies, task delegation routines, and handoff logic between distinct specialist agents.

AI-103T00-A Phase 3: Language and Speech Integration

Focus: Processing textual and acoustic signals structurally within software applications.

- **Lab 12: Analyze text**
  - *Design Focus:* Traditional natural language workflows including sentiment parsing, keyphrase extraction, and entity recognition.
- **Lab 13: Develop A Text Analysis Agent**
  - *Design Focus:* Wrapping natural language processing rules inside a dedicated, loop-based structural text analysis agent.
- **Lab 14: Use speech-capable generative AI models**

  - *Design Focus:* Integrating multi-modal audio endpoints capable of processing and generating speech streams.

- ➡ **Lab 15: Recognize and synthesize speech**
  - *Design Focus:* Designing speech-to-text (STT) and text-to-speech (TTS) pipelines over standard REST or SDK mechanisms.
- ➡ **Lab 16: Use Azure Speech in an agent**
  - *Design Focus:* Giving a structural agent voice features by passing local acoustic micro-buffers down to the backend model.
- ➡ **Lab 17: Develop a Voice Live agent**
  - *Design Focus:* Configuring low-latency, full-duplex bi-directional streaming for real-time human-agent audio conversations.

🗺 AI-103T00-A Phase 4: Multimodal Vision and Content Understanding

Focus: Processing visual artifacts, structural document parsing, and custom extraction schemas.

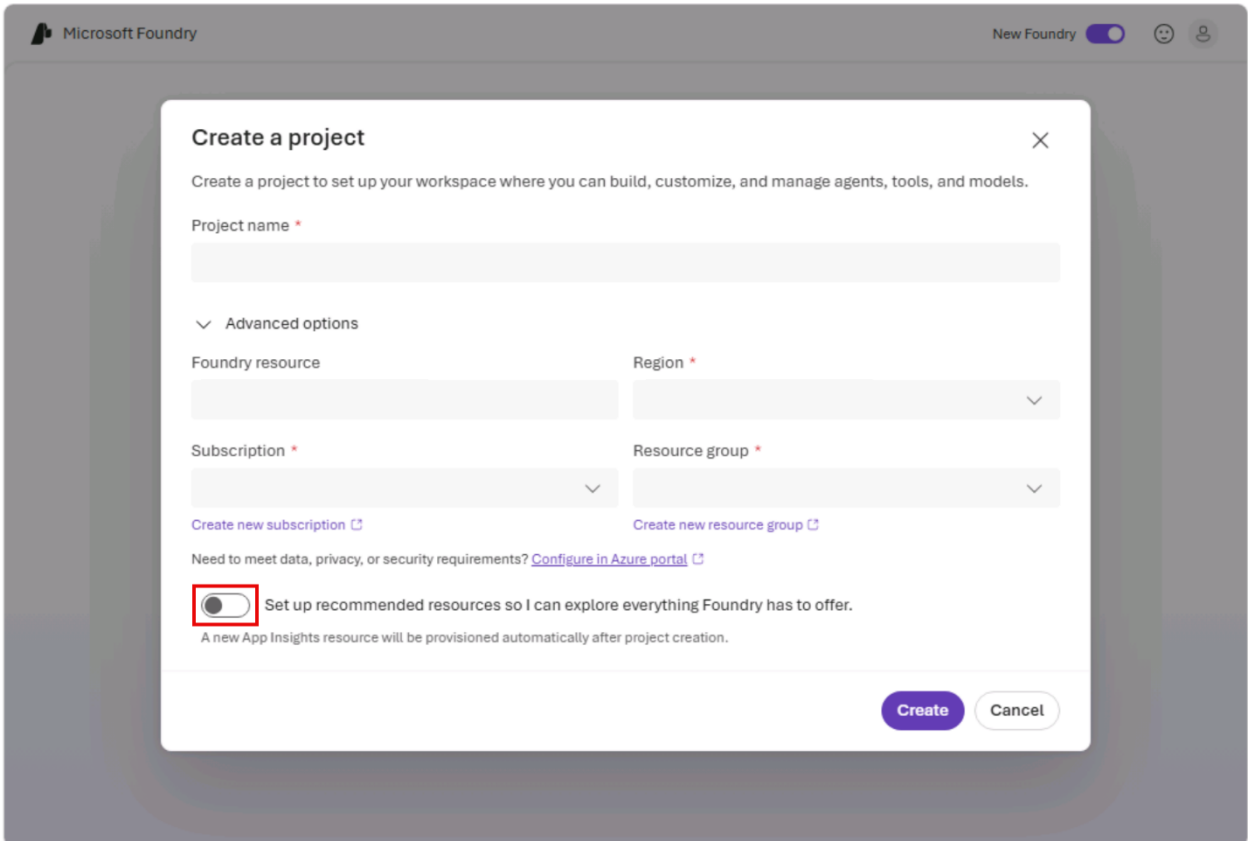
- ➡ **Lab 18: Develop a vision-chat app**
  - *Design Focus:* Submitting mixed payloads containing high/low resolution imagery along with text tokens to multi-modal vision models.
- ➡ **Lab 19: Generate images with AI**
  - *Design Focus:* Implementing text-to-image workflows using DALL-E models, handling aspect ratios, quality profiles, and variations.
- ➡ **Lab 20: Analyze images with Azure Content Understanding**
  - *Design Focus:* Setting up layout engines to detect structure, extract tables, and perform optical character recognition (OCR).
- ➡ **Lab 21: Extract information from multimodal content (Optional)**
  - *Design Focus:* Advanced parsing workflows that read data fields from messy real-world visual assets like receipts, IDs, and mixed forms.
- ➡ **Lab 22: Develop a Content Understanding client application**
  - *Design Focus:* Standard software pattern design for consuming custom extractor endpoints programmatically within native client apps.

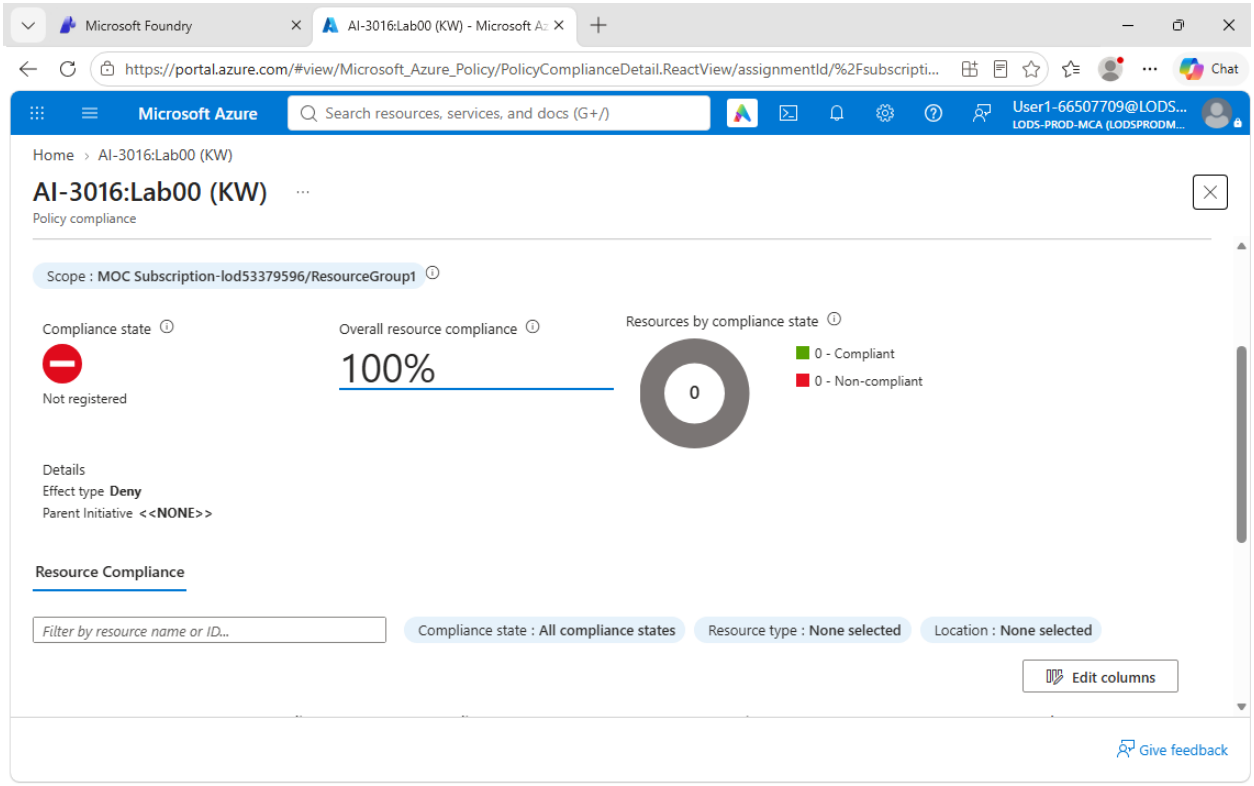
Very Loose Lab Notes to Erase Create Project

Prepare for an AI development project

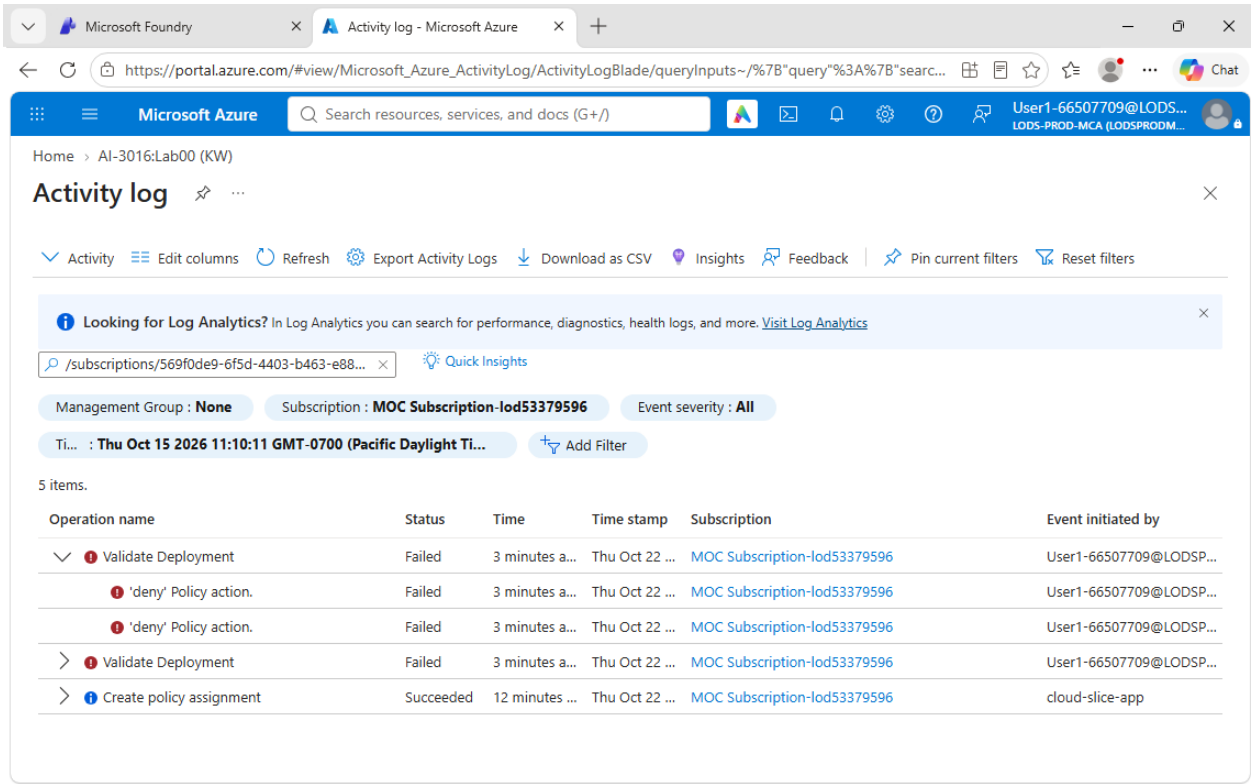
Instructions Resources

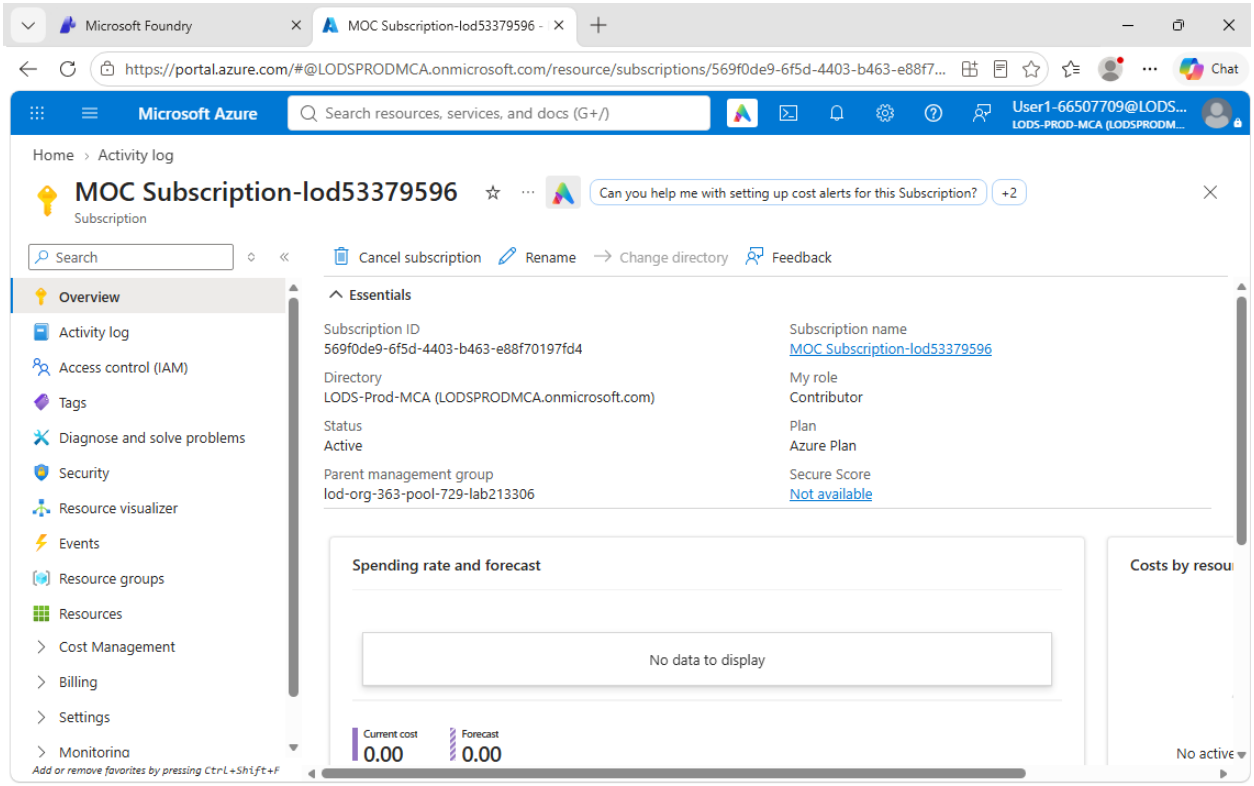
When creating a project using the New Foundry setting, ensure you uncheck the option to **Set up recommended resources so I can explore everything Foundry has to offer** and create resources following the instructional steps:





Prepare for an AI development project test cloud slice





Copilot sez:

This error means an **Azure Policy** is preventing the resource from being created. The policy named **AI-3016:Lab00 (KW)** is assigned to your lab/subscription and only allows specific resource types, names, locations, or configurations. When your resource doesn't match the policy rules, Azure blocks the deployment with a *RequestDisallowedByPolicy* error. [learn.microsoft.com], [learn.microsoft.com]

Common causes in AI-3016 lab environments

1. **Incorrect resource name**
    - Many Microsoft Learn/Skillable labs require a specific naming convention.
    - For AI-3016 labs, custom names are often blocked by policy. [github.com]
2. **Wrong Azure region**
    - The lab may allow only specific regions (for example, East US or Sweden Central).
    - Creating resources in another region can trigger a policy violation. [learn.microsoft.com]
3. **Unsupported resource type or SKU**
    - The policy may allow only certain Azure AI resources or pricing tiers. [learn.microsoft.com], [learn.microsoft.com]
4. **Creating resources outside the designated resource group**

- Lab environments frequently restrict deployments to a pre-created resource group. [learn.microsoft.com]

How to identify the exact reason

In the Azure portal:

1. Open **Resource Group → Deployments**.
2. Select the failed deployment.
3. Open **Operation details**.
4. Look for:
   - **Policy Assignment**
   - **Policy Definition**
   - **Evaluation Details**

Azure typically shows which rule was violated. [learn.microsoft.com]

You can also check:

Azure Portal

Policy -> Compliance

Policy -> Assignments

and locate the **AI-3016:Lab00 (KW)** assignment. [learn.microsoft.com], [learn.microsoft.com]

If this is a Microsoft Learn / Skillable lab

The most common fix is to:

- Use the exact resource name specified in the lab instructions.
- Use the provided resource group.
- Use the required region.
- Use the required SKU/tier.
- Do not create additional resources outside the lab steps. [github.com]

If you have administrator access

Review or edit the policy assignment:

Azure Portal -> Policy -> Assignments

Find **AI-3016:Lab00 (KW)** and check which resource types, locations, or names are allowed. Then either:

- Modify the deployment to comply, or

- Update the policy/exemption if permitted. [learn.microsoft.com], [learn.microsoft.com]

If you can share the **full deployment error details** (especially the Policy Violation section), I can pinpoint the exact setting causing the block.

Error part 2 – Be sure Project is named & Correct Region

Yes. Even if the error appears when signing in to **Azure AI Foundry**, you can usually investigate the underlying Azure resources in the **Azure portal**, assuming your lab account has sufficient permissions.

Check the resource group

1. Open the **Azure Portal** (portal.azure.com).
2. Search for **Resource groups**.
3. Look for the resource group provided by your lab instructions.
4. Open it and review:
    - **Deployments** (for failed deployments)
    - **Resources** (AI Foundry Hub, Project, Azure AI Services, Storage Account, etc.)
    - **Activity Log** (often shows policy-related failures) [learn.microsoft.com]

If you cannot find any resource groups

In many AI-3016 lab environments, the policy block occurs **before the Foundry project or its backing resources are created**. In that case:

1. Go to **Subscriptions** in Azure Portal.
2. Select your lab subscription.
3. Open **Activity Log**.
4. Filter for **Failed** operations around the time of the error.
5. Open the failed event and look for:
    - RequestDisallowedByPolicy
    - Policy assignment name
    - Policy definition details [learn.microsoft.com]

Since the policy name is AI-3016:Lab00 (KW)

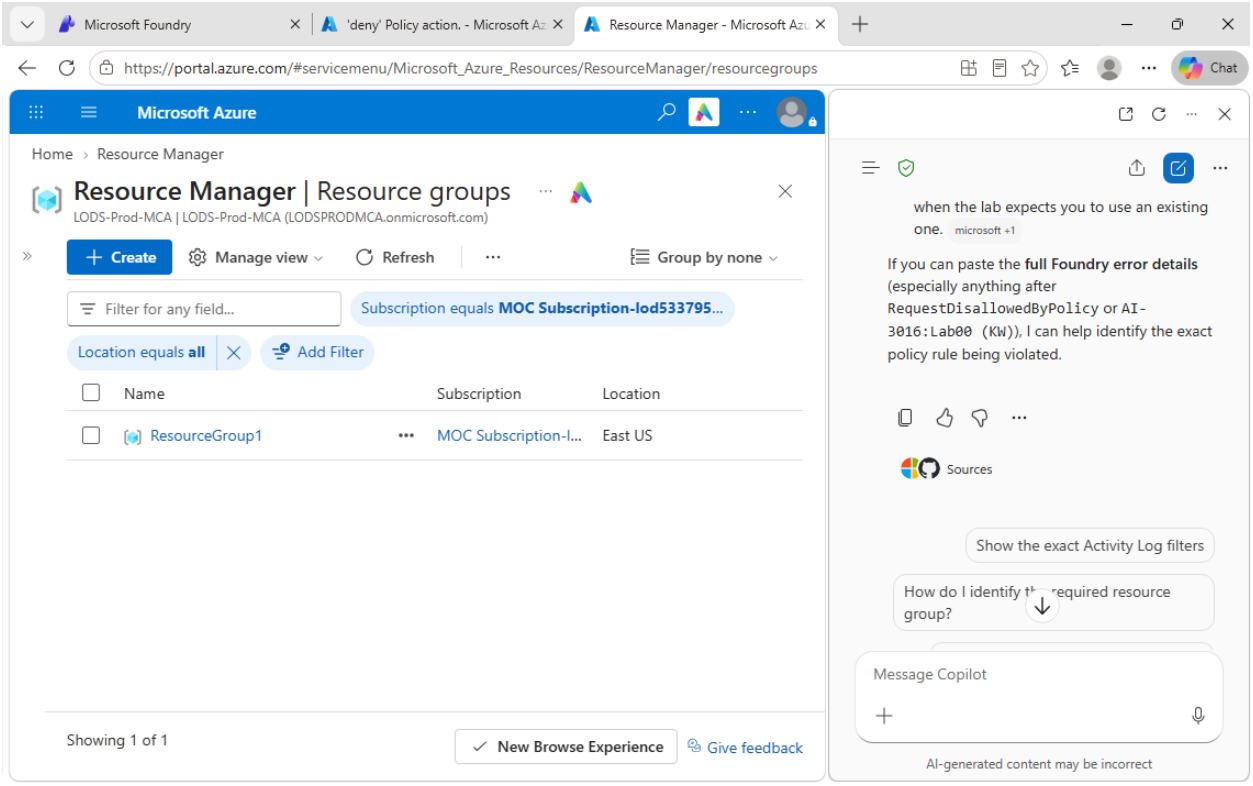
This looks very much like a Microsoft Learn/Skillable training policy that restricts what can be created. Similar AI-3016 lab environments have required specific project names and configurations rather than arbitrary values. [github.com]

Another possibility

If the error occurs immediately upon creating or opening a Foundry project, verify:

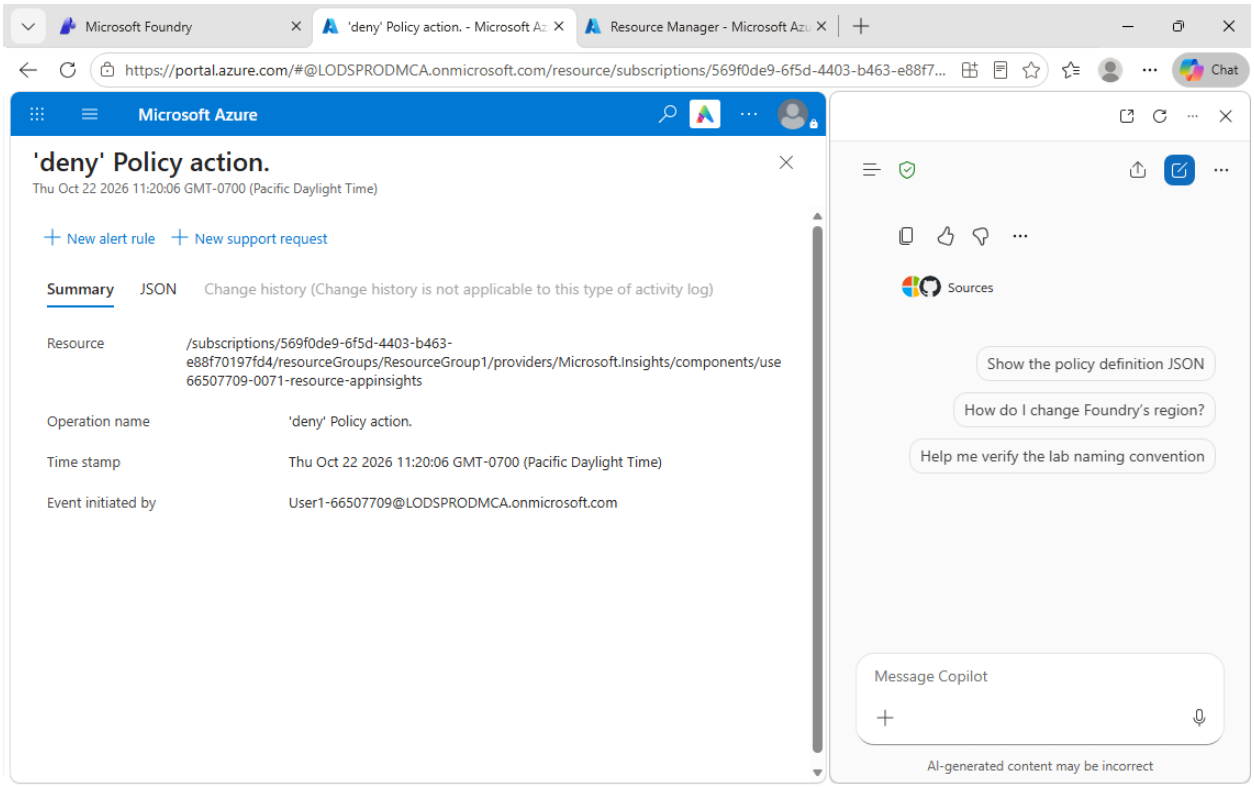
- You're using the exact **Hub/Project name** specified by the lab.
- You're in the **correct Azure subscription**.
- You're creating the project in the **required region**.
- You're not creating a new resource group when the lab expects you to use an existing one. [learn.microsoft.com], [github.com]

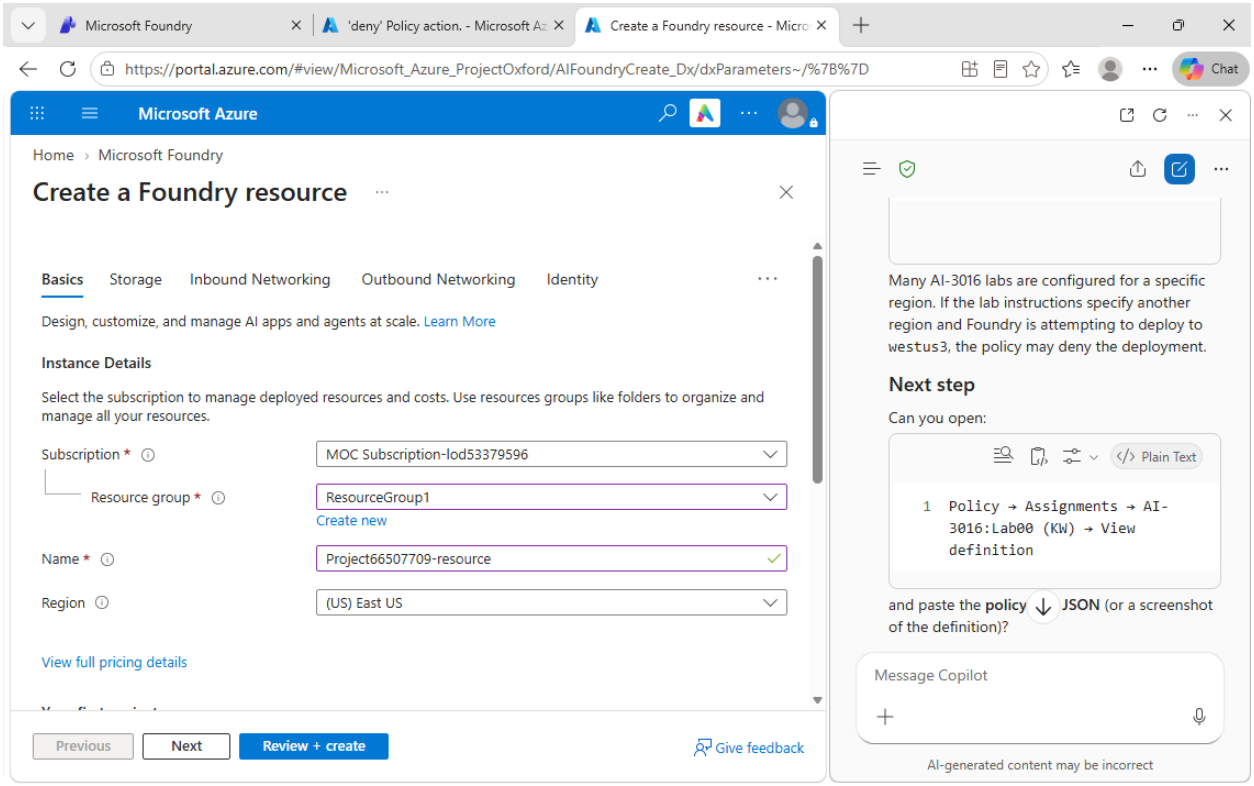
If you can paste the **full Foundry error details** (especially anything after RequestDisallowedByPolicy or AI-3016:Lab00 (KW)), I can help identify the exact policy rule being violated.

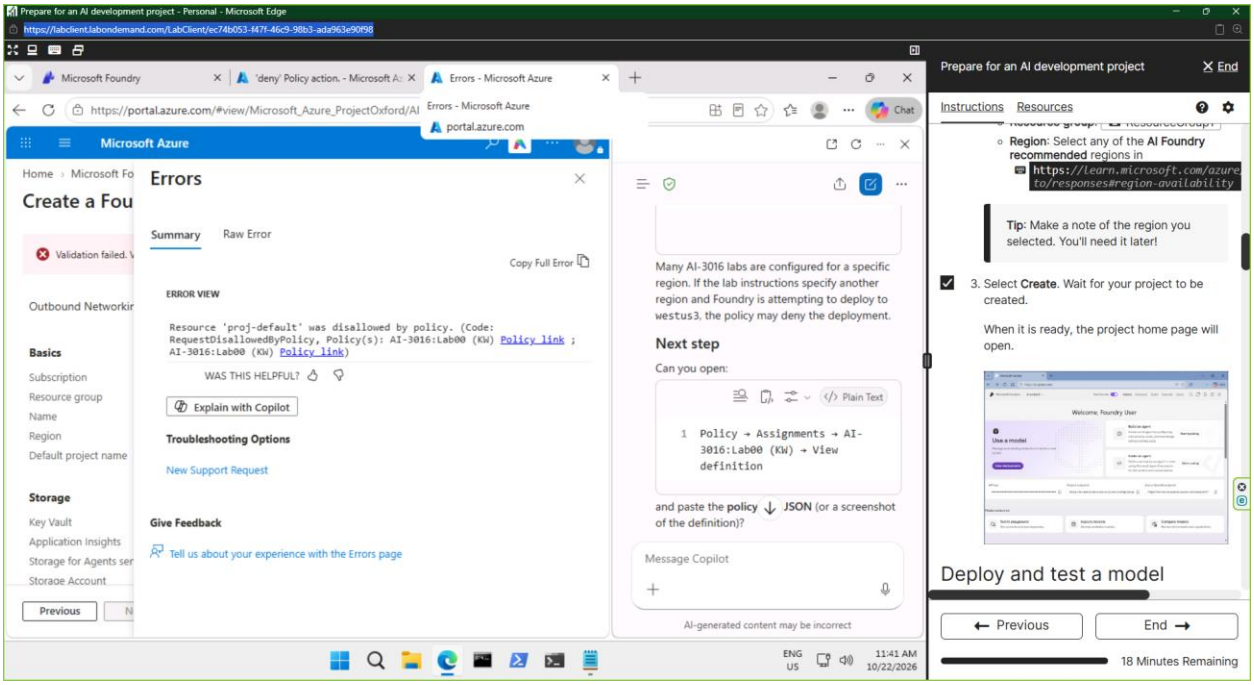


Things 2 Check w Tim Screenshots

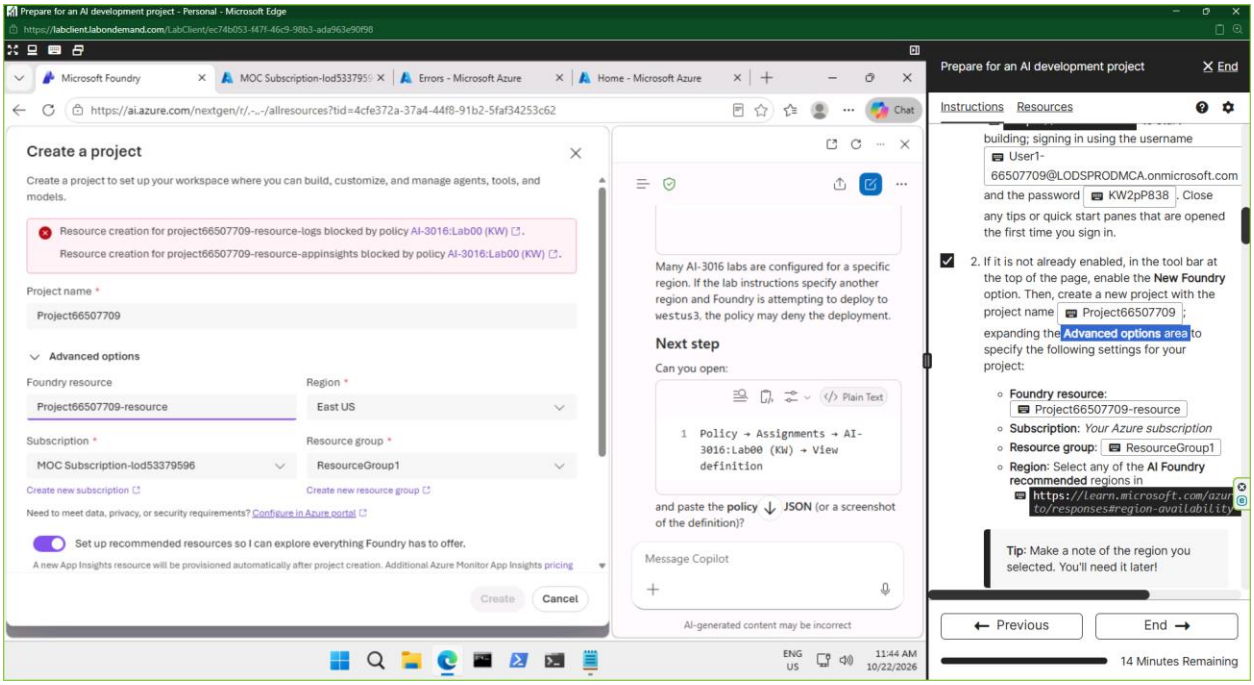
The details you posted are enough to identify the likely cause.



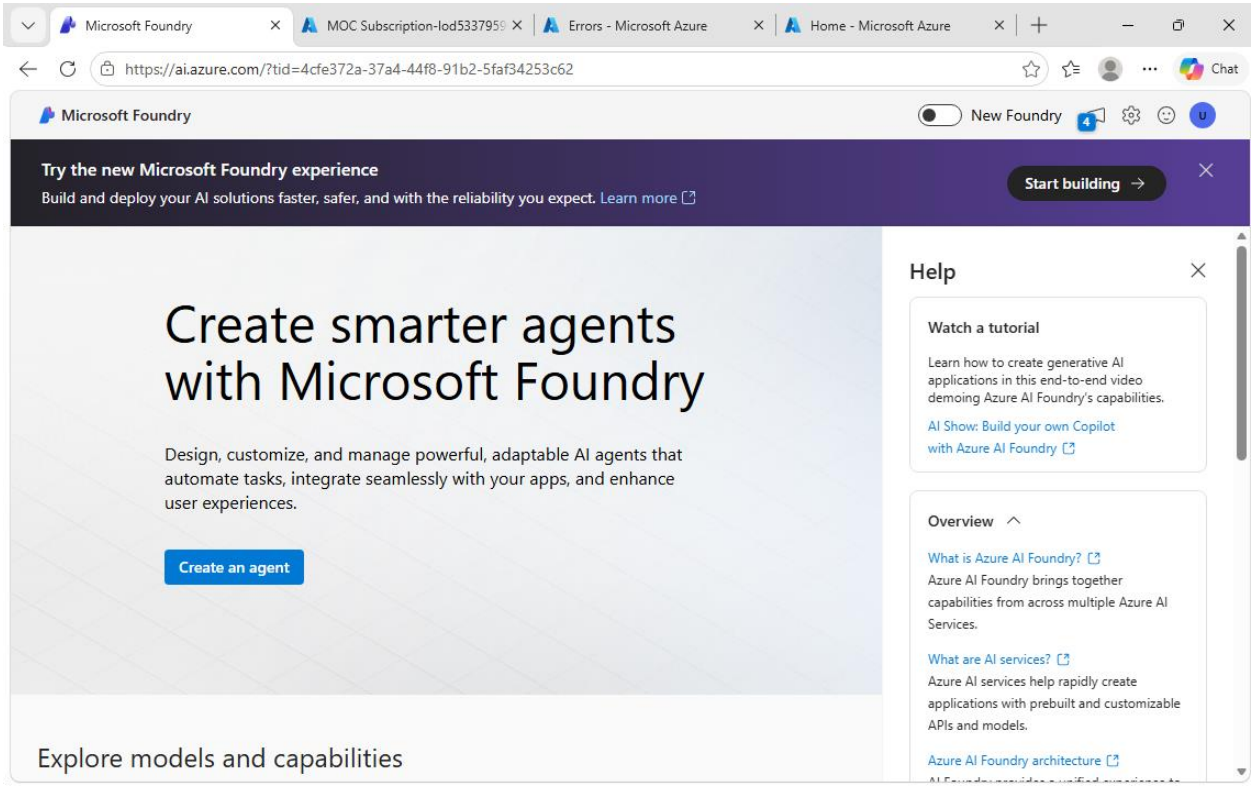


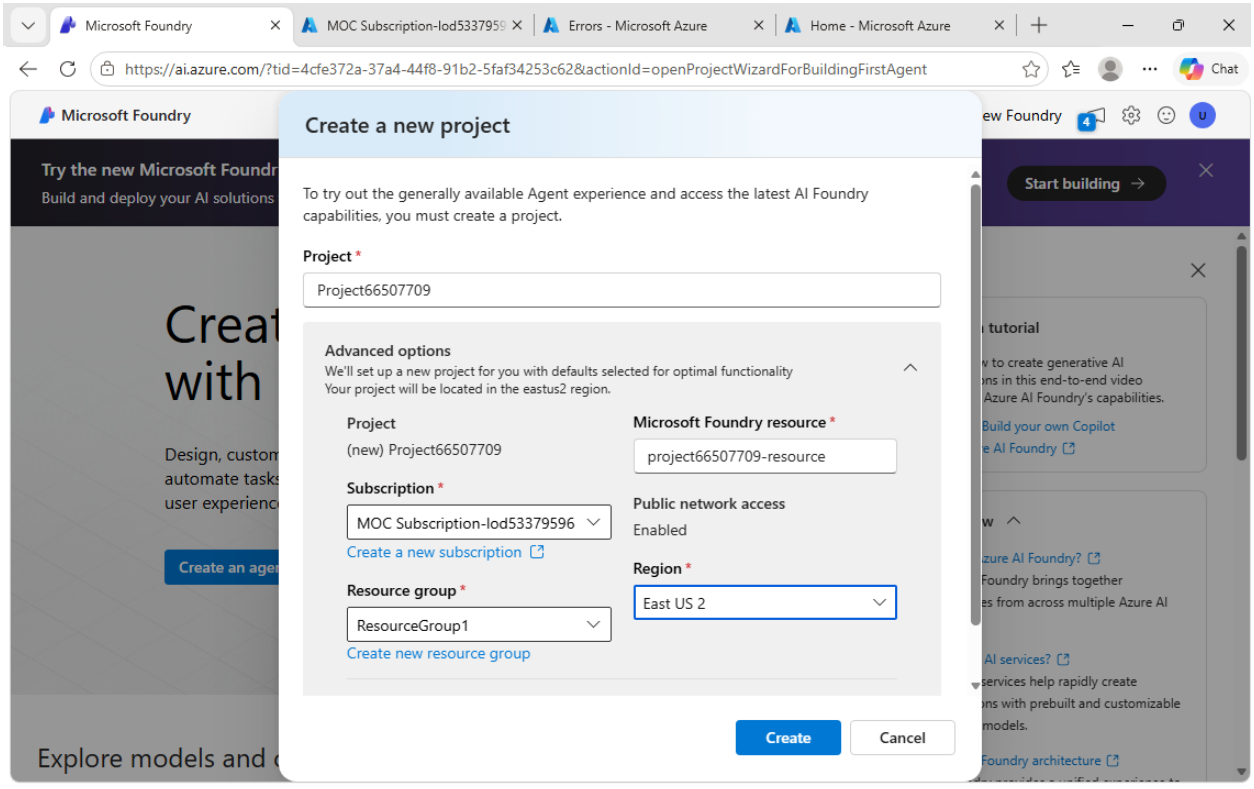


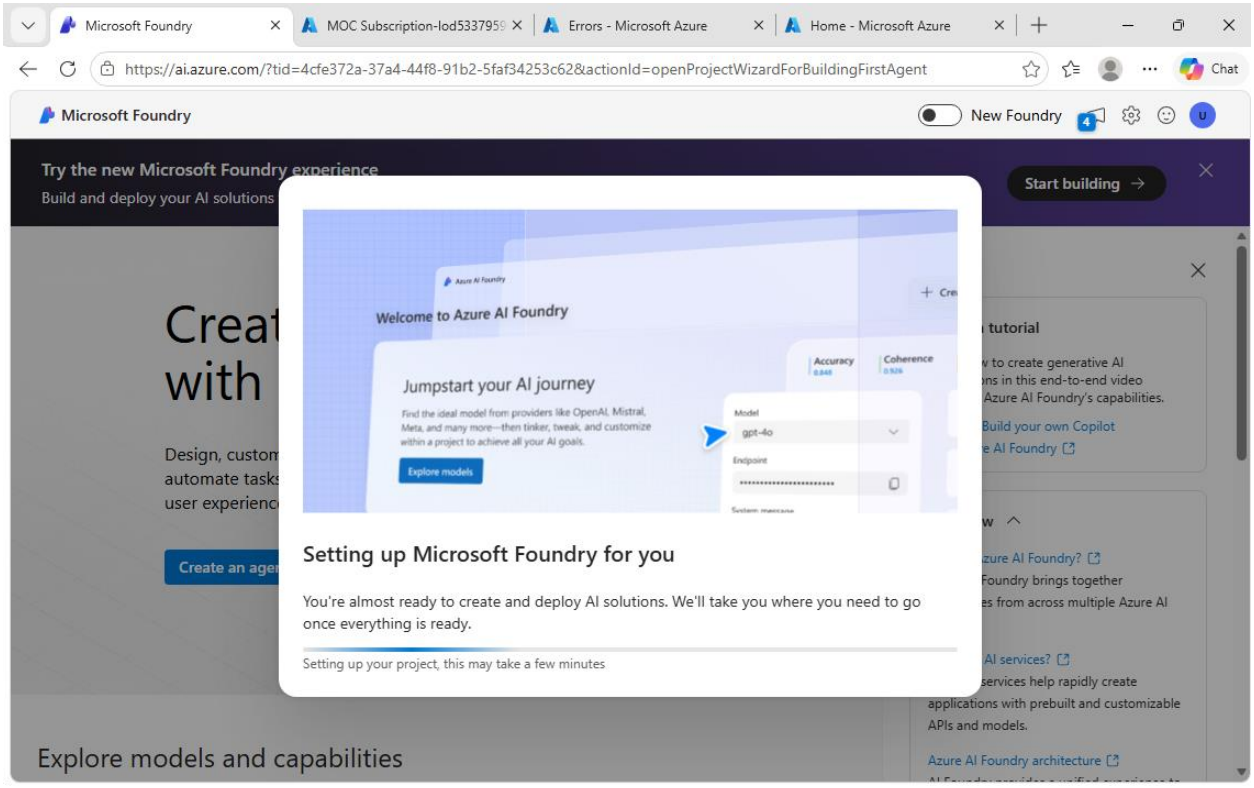
Settings get rid of errors?

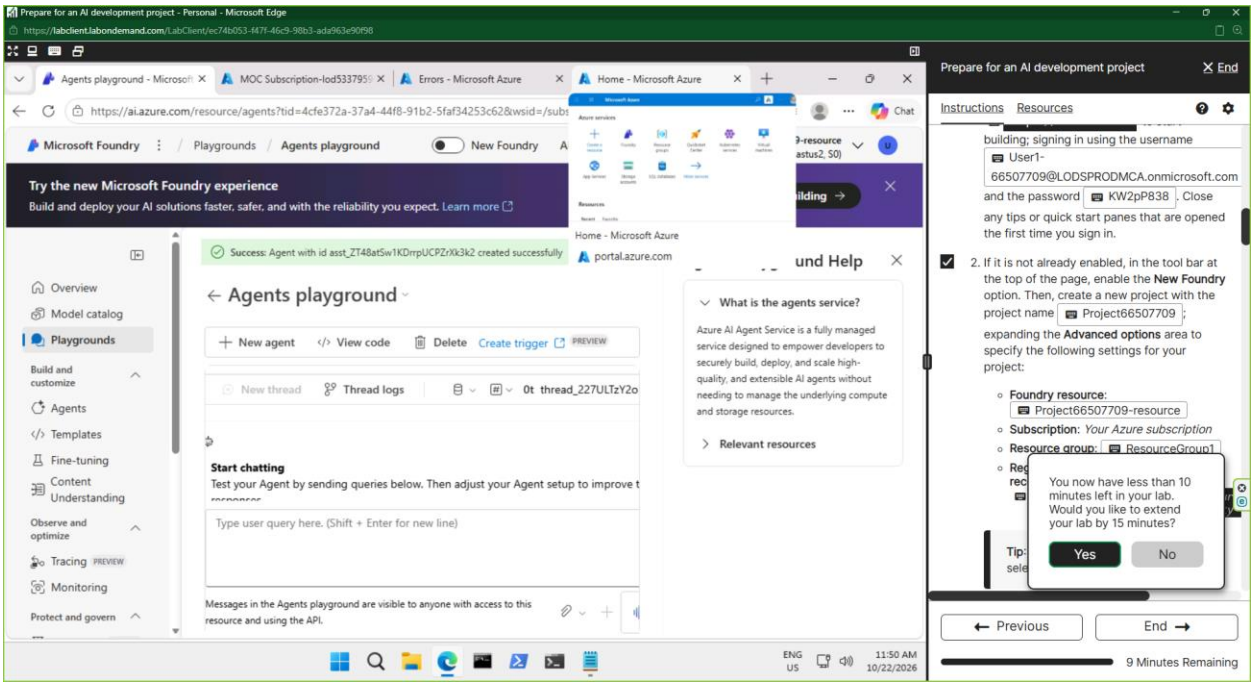


Old Foundry

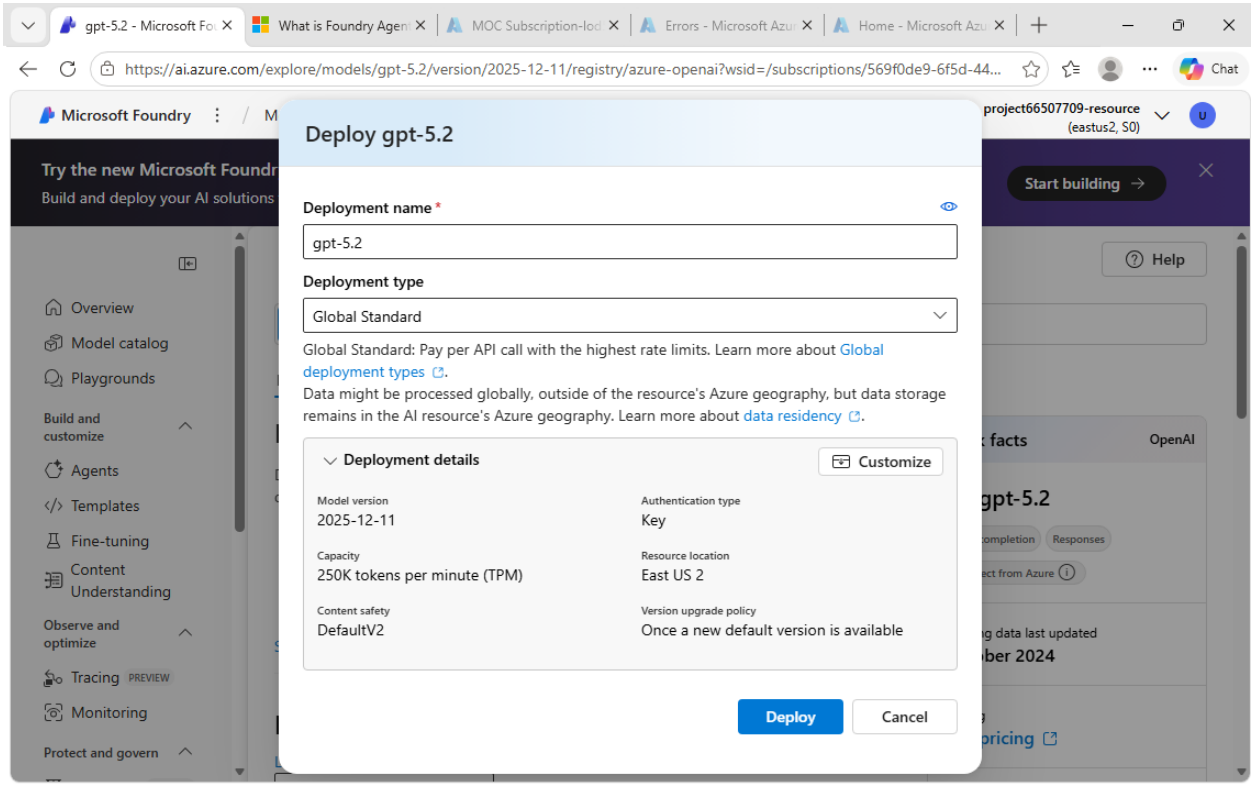

Microsoft Foundry
MOC Subscription-lod5337959
Errors - Microsoft Azure
Home - Microsoft Azure
https://ai.azure.com/?tid=4cfe372a-37a4-44f8-91b2-5faf34253c62
Chat
Microsoft Foundry
New Foundry
Try the new Microsoft Foundry experience
Build and deploy your AI solutions faster, safer, and with the reliability you expect. Learn more
Start building
Create smarter agents with Microsoft Foundry
Design, customize, and manage powerful, adaptable AI agents that automate tasks, integrate seamlessly with your apps, and enhance user experiences.
Create an agent
Explore models and capabilities
Help
Watch a tutorial
Learn how to create generative AI applications in this end-to-end video demoing Azure AI Foundry's capabilities.
AI Show: Build your own Copilot with Azure AI Foundry
Overview
What is Azure AI Foundry?
Azure AI Foundry brings together capabilities from across multiple Azure AI Services.
What are AI services?
Azure AI services help rapidly create applications with prebuilt and customizable APIs and models.
Azure AI Foundry architecture


Microsoft Foundry
MOC Subscription-lod5337959
Errors - Microsoft Azure
Home - Microsoft Azure
https://ai.azure.com/?tid=4cfe372a-37a4-44f8-91b2-5faf34253c62&actionId=openProjectWizardForBuildingFirstAgent
Chat
Create a new project
To try out the generally available Agent experience and access the latest AI Foundry capabilities, you must create a project.
Project *
Project66507709
Advanced options
We'll set up a new project for you with defaults selected for optimal functionality
Your project will be located in the eastus2 region.
Project
(new) Project66507709
Subscription *
MOC Subscription-lod53379596
Create a new subscription
Resource group *
ResourceGroup1
Create new resource group
Microsoft Foundry resource *
project66507709-resource
Public network access
Enabled
Region *
East US 2
Create
Cancel

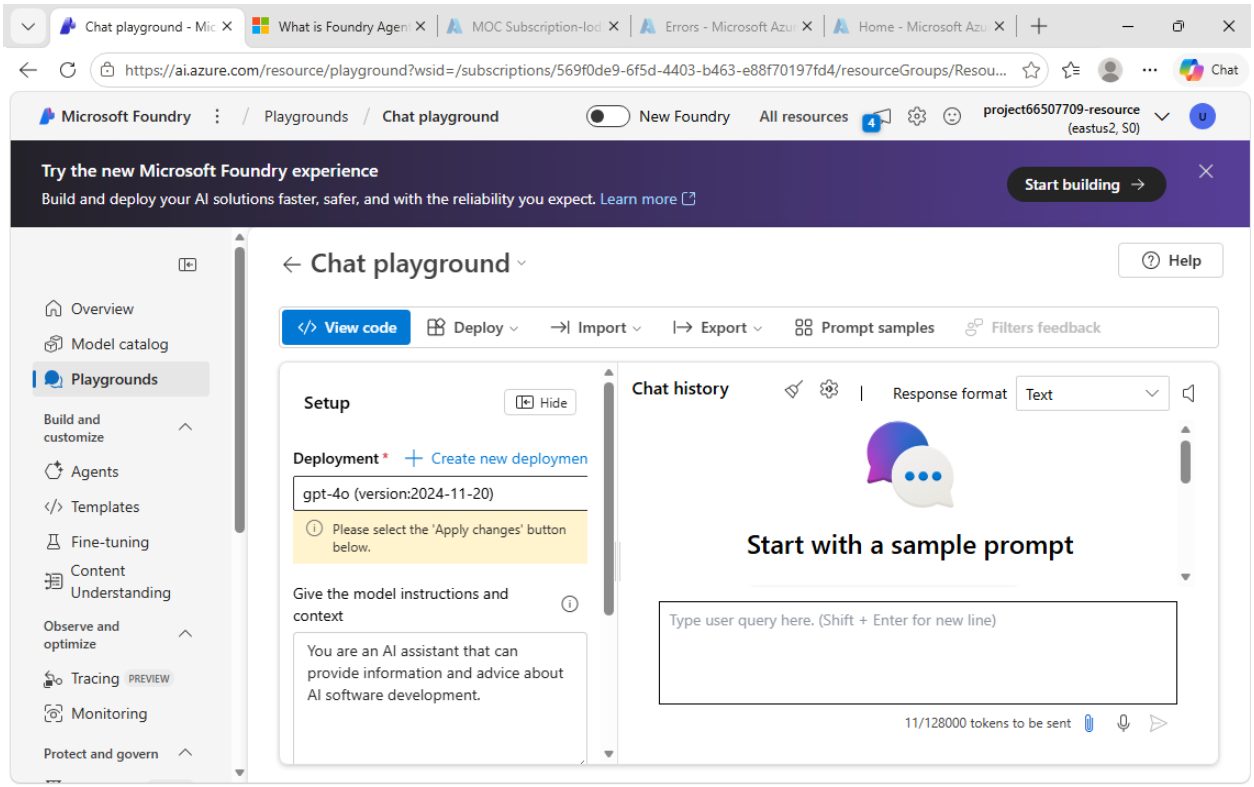




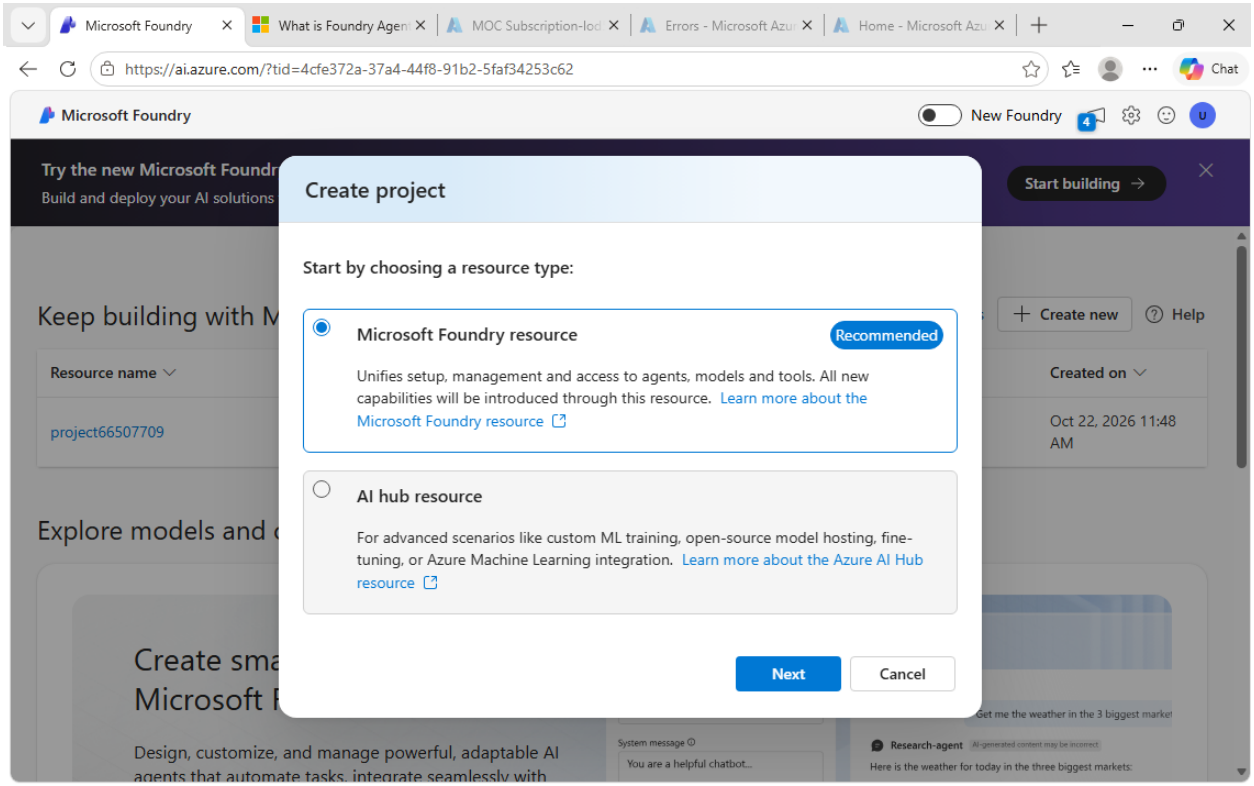
[Quickstart - Create a new Foundry Agent Service project (classic) - Microsoft Foundry (classic) portal | Microsoft Learn]

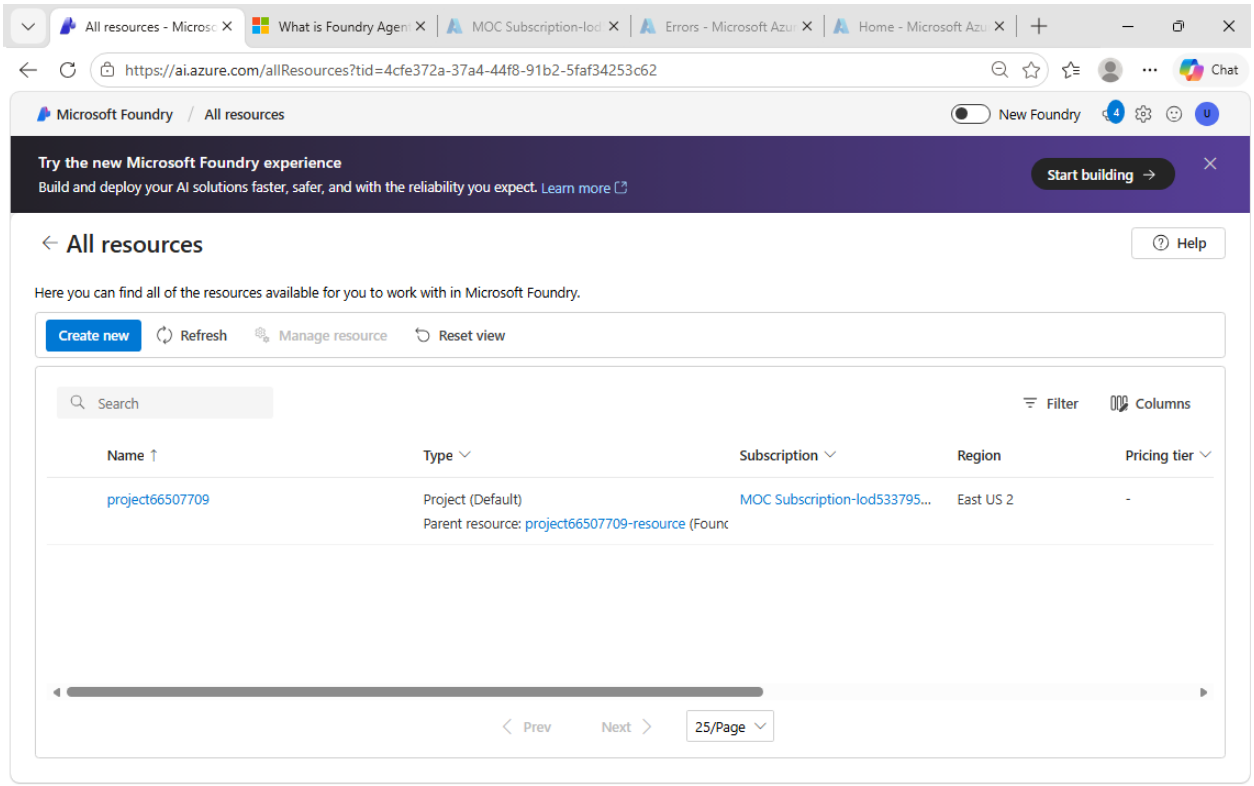


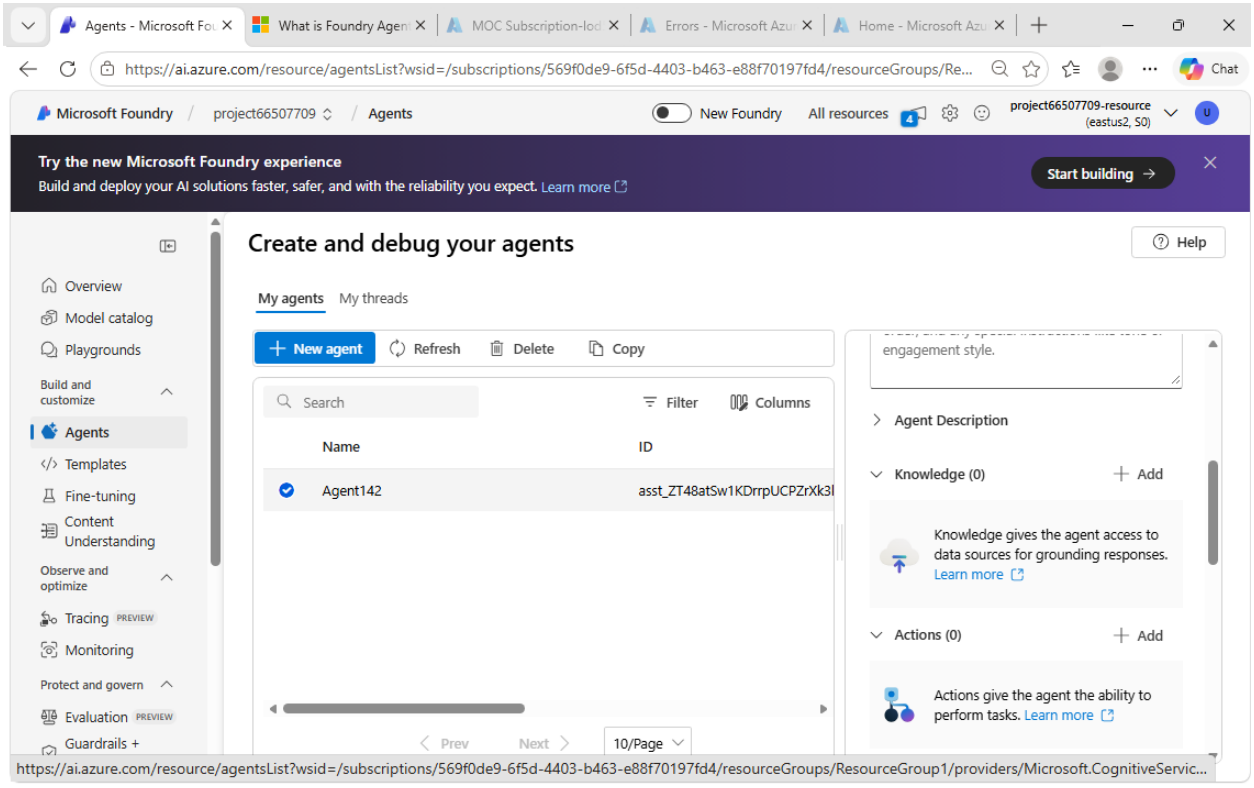
[How to add and manage data in your Microsoft Foundry hub-based project (classic) - Microsoft Foundry (classic) portal | Microsoft Learn]



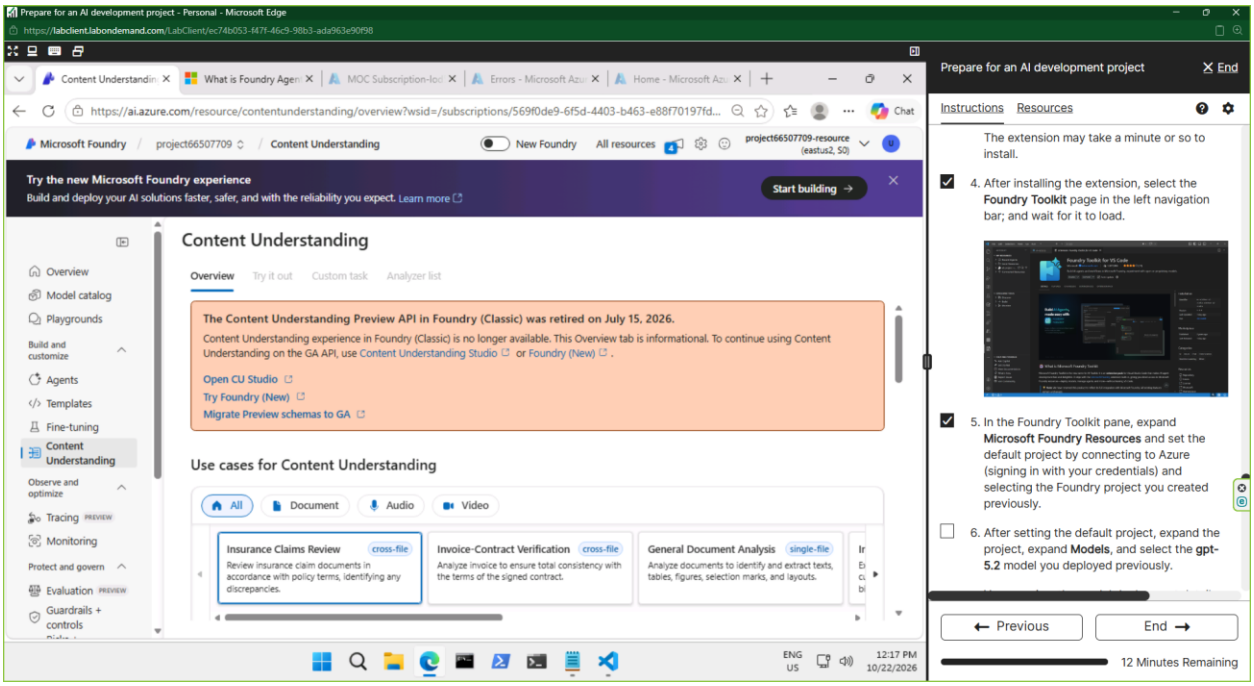
Browse Foundry Portal & Tools

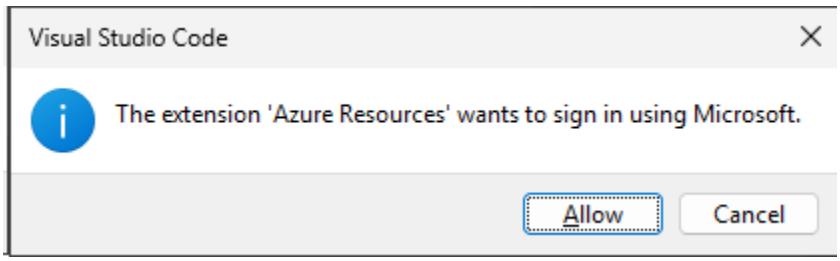




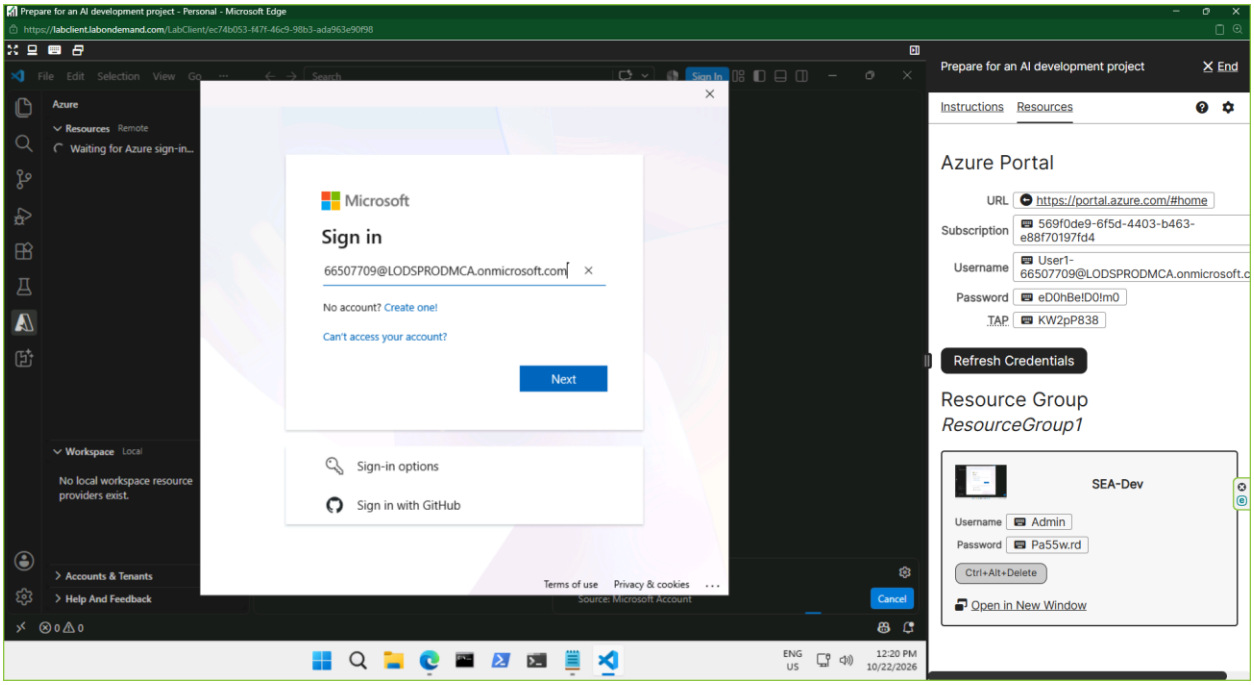


Retired Content Understanding 07/15/2026





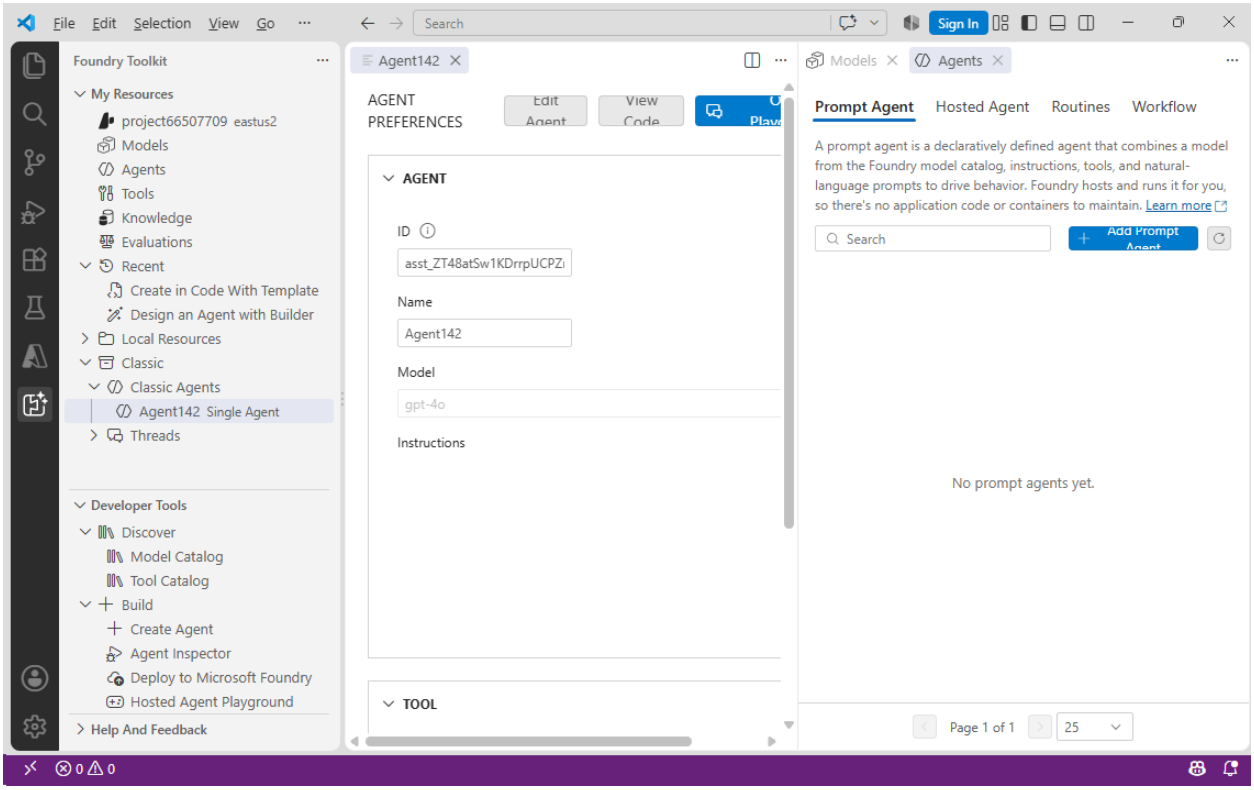
.



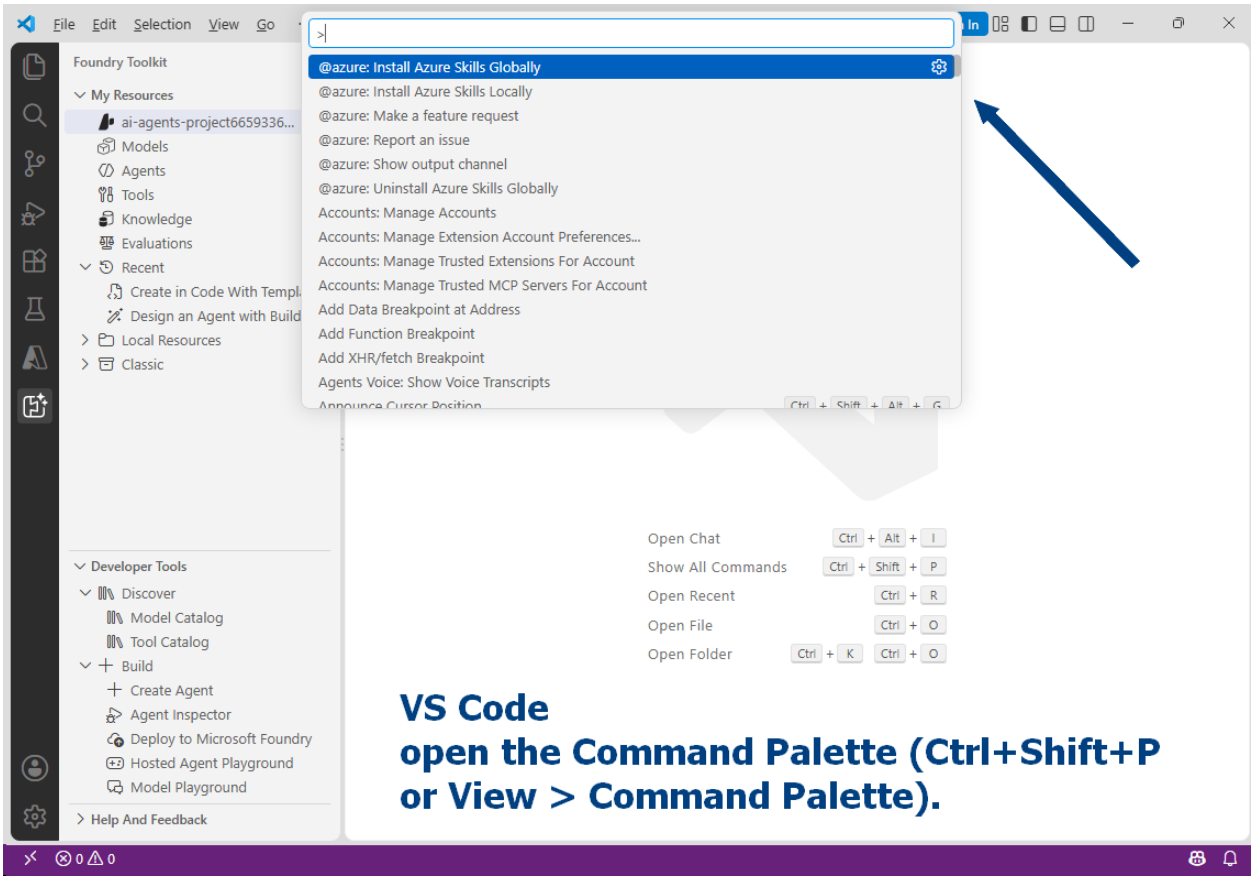
VS Code – Foundry Extension – Choose Project

Classic Agent in VS Code

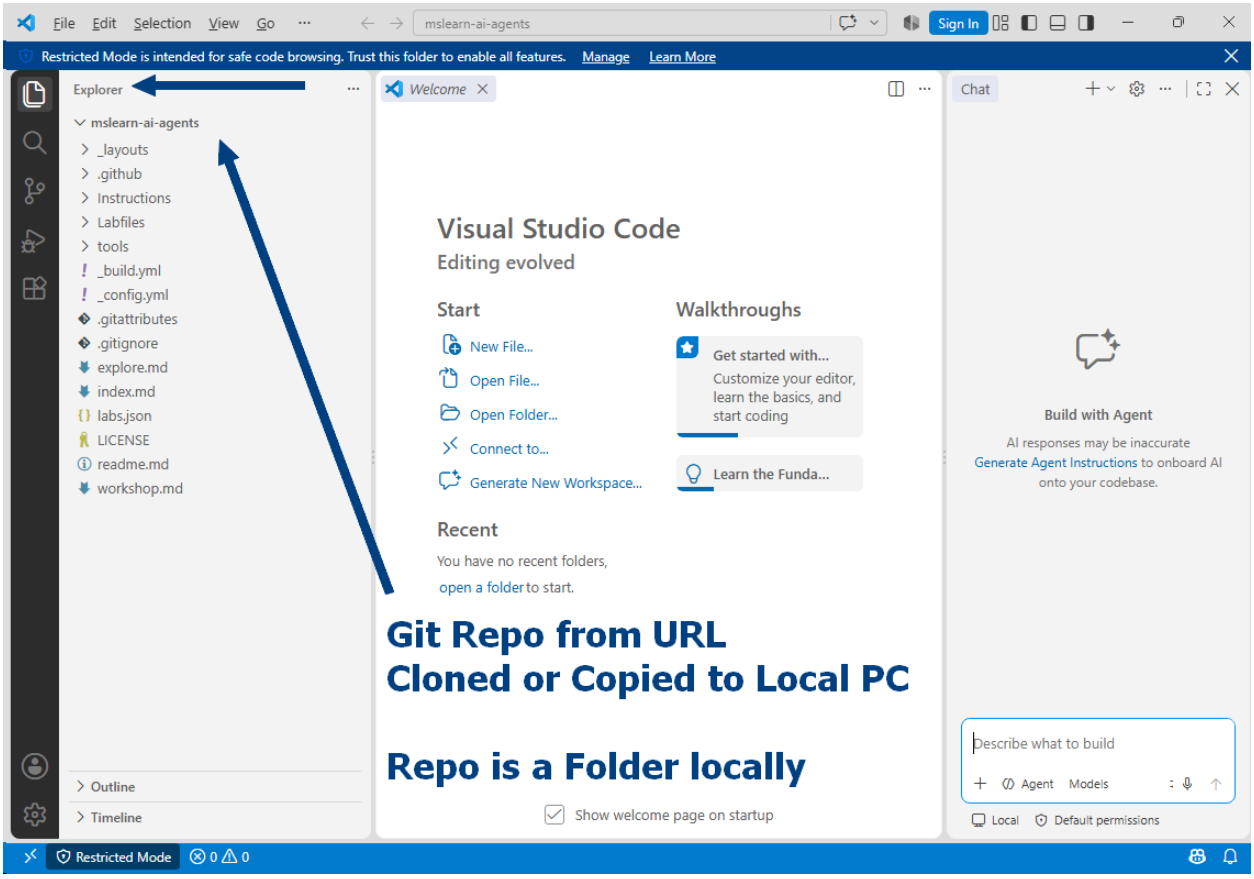
Foundry Toolkit for VS Code extension to create an agent that can use Model Context Protocol (MCP) server tools to access external data sources and APIs. The agent will be able to retrieve up-to-date information and interact with custom services through MCP tools.



Command Palette



Open/Clone Git Repo



New Foundry vs Classic

Microsoft Foundry (New) and Microsoft Foundry (Classic) (formerly Azure AI Studio / Azure AI Foundry) represent two different architectural generations and user interfaces accessible via a portal toggle at ai.azure.com. [1, 2]

Key Differences

- **Underlying Architecture:**
  - **Classic:** Uses an older hub-based architecture (`Microsoft.MachineLearningServices`) that requires managing mandatory sibling and backend Azure resources.
  - **New:** Streamlines into a native project model (`Microsoft.CognitiveServices`) with control-plane and data-plane separation, removing the need for an intermediate hub. [1, 2, 3]
- **Project Models:**
  - **Classic:** Relies on Hub + AI Project structures.
  - **New:** Focuses strictly on unified Foundry Projects with simplified resource management, monitoring, and RBAC (Role-Based Access Control). [1]
- **Agent Capabilities & Workflows:**

    - **Classic:** Limited primarily to single-agent flows, Prompt Flow, and AutoML in hub-based projects.
    - **New:** Introduces multi-agent visual workflows, long-term memory, Foundry IQ knowledge integration, and direct publishing to Microsoft 365 and Teams. [1, 2, 3]
- **Tools and Integrations:**
    - **Classic:** Traditional Azure AI services and endpoint connections.
    - **New:** Features a unified Tools tab acting as an entry point for 1,400+ tool catalogs, Model Context Protocol (MCP) servers, and Agent-to-Agent (A2A) endpoints. [1, 2, 3]
- **Lifecycle & Status:**
    - **Classic:** Deprecated as of July 15, 2026, though older hub deployments remain accessible for legacy support.
    - **New:** The active default platform where all modern features, model updates, and active investments land. [1, 2, 3]

log shows for Deny Create error: Incorrect Project Name/Region

The deployment was blocked by a **Deny** Azure Policy:

Policy Display Name: AI-3016:Lab00 (KW)

Effect: deny

Scope: ResourceGroup1

Location: westus3

The resource involved is:

Microsoft.Insights/components

user1-66507709-0071-resource-appinsights

which is an **Application Insights** resource. The failure occurred during deployment validation (Microsoft.Resources/deployments/validate/action) before the resource was actually created. [learn.microsoft.com]

Why this is happening in AI Foundry

When Azure AI Foundry creates a Hub or Project, it automatically deploys supporting resources such as:

- Azure AI Services
- Storage Account
- Key Vault
- Application Insights

If the lab policy doesn't allow one of those resources, the entire Foundry project creation fails. This is exactly what your log indicates: the policy blocked the creation of the App Insights resource.

Check the actual policy rule

In Azure Portal:

1. Go to **Policy**.
2. Select **Assignments**.
3. Find **AI-3016:Lab00 (KW)**.
4. Open it.
5. Click **View definition**.

Look for rules involving:

"type": "Microsoft.Insights/components"

or:

"location"

or naming requirements matching:

user1-66507709-xxxx-resource

Most likely cause

I notice the blocked resource name is:

user1-66507709-0071-resource-appinsights

while your earlier error mentioned:

user1-66507709-0239-resource

Many Microsoft Learn / Skillable labs enforce strict naming patterns and only allow resources matching the lab instance number. Similar AI-3016 lab issues have been reported where custom names were blocked by policy. [github.com]

Can you see the detailed policy condition?

Yes. Try either:

Policy → Assignments → AI-3016:Lab00 (KW) → View definition

or

Policy → Compliance → AI-3016:Lab00 (KW)

The policy JSON should reveal exactly what is being denied.

One thing I'd check immediately

The log shows:

resourceLocation = westus3

Many AI-3016 labs are configured for a specific region. If the lab instructions specify another region and Foundry is attempting to deploy to westus3, the policy may deny the deployment.

Next step

Can you open:

Policy → Assignments → AI-3016:Lab00 (KW) → View definition

and paste the **policy rule JSON** (or a screenshot of the definition)?

That will tell us precisely whether the block is due to:

- region (westus3)
- resource type (Microsoft.Insights/components)
- naming convention
- allowed resource list

and I can identify the exact fix.

Lab Test Cloud Slice/Select New Foundry vs Classic

In this exercise, you use Microsoft Foundry portal to create a project, ready to build an AI solution.

This exercise takes approximately **30** minutes.

Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An active Azure subscription
- Visual Studio Code installed
- Python version **3.13.xx** installed*
- Git installed and configured
- Azure CLI installed

* Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

Create a Microsoft Foundry project

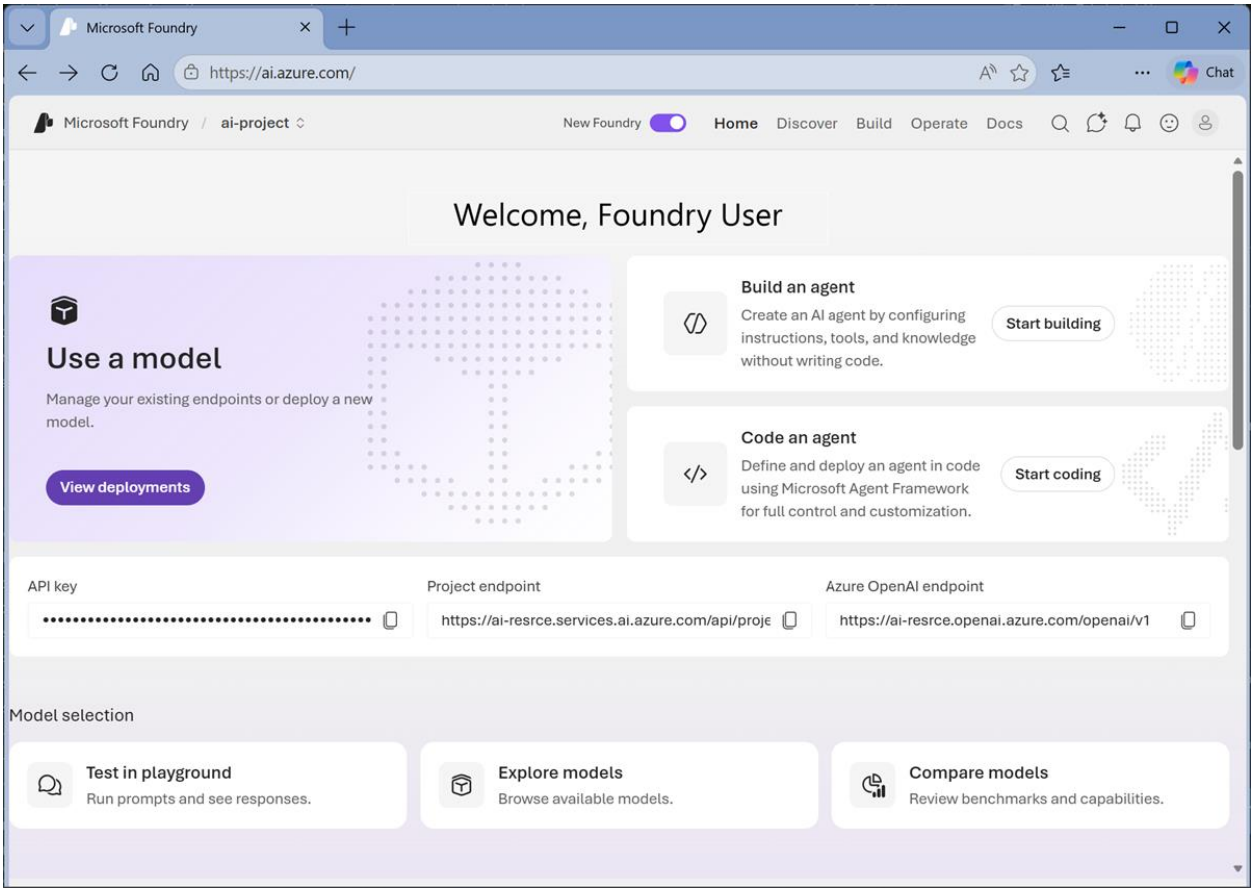
Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

1. In a web browser, open the Microsoft Foundry portal at https*://ai.azure.com* to start building; signing in using the username User1-66505261@LODSPRODMCA.onmicrosoft.com and the password =x86dmrk. Close any tips or quick start panes that are opened the first time you sign in.
2. If it is not already enabled, in the tool bar at the top of the page, enable the **New Foundry** option. Then, create a new project with the project name Project66505261; expanding the **Advanced options** area to specify the following settings for your project:
    - **Foundry resource**: Project66505261-resource
    - **Subscription**: *Your Azure subscription*
    - **Resource group**: ResourceGroup1
    - **Region**: Select any of the **AI Foundry recommended** regions in https*://learn.microsoft.com/azure/foundry/openai/how-to/responses#region-availability*

Tip: Make a note of the region you selected. You'll need it later!

3. Select **Create**. Wait for your project to be created.

When it is ready, the project home page will open.

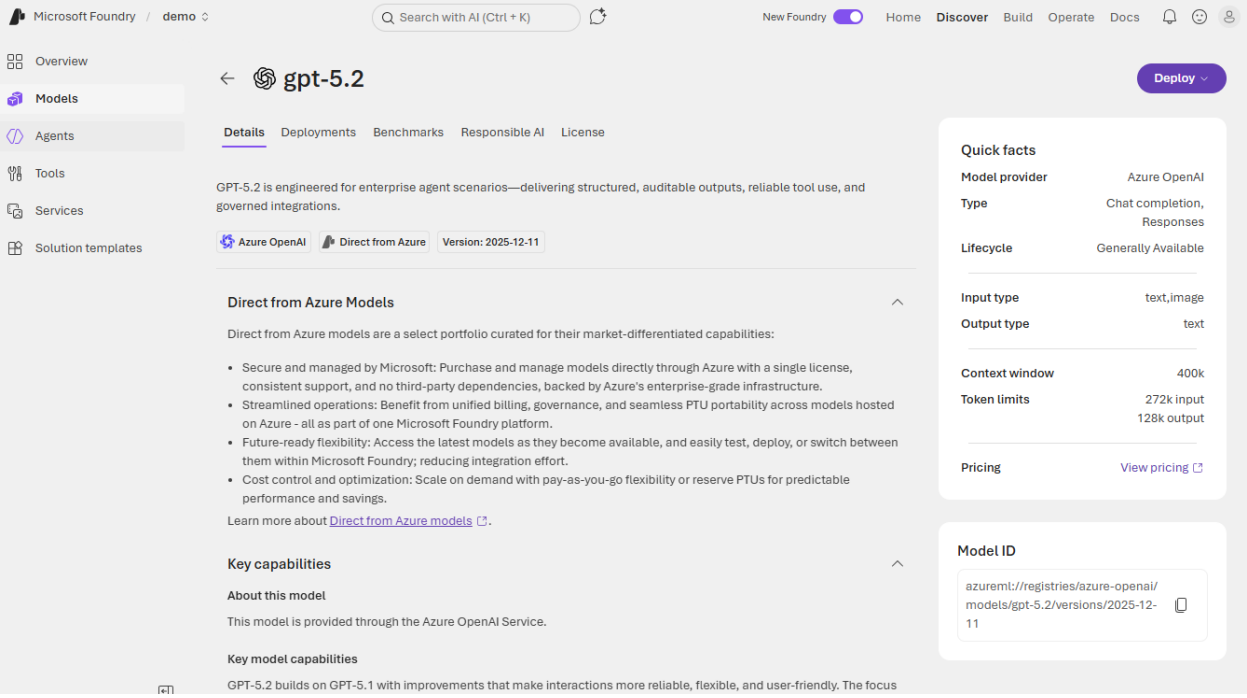


Deploy and test a model

At the core of any generative AI project, there's at least one generative AI model.

1. Now you're ready to explore models. On the **Discover** page, select the **Models** tab to view the Microsoft Foundry model catalog.
2. Search for the gpt-5.2 model, and then select it in the search results to view its model card.

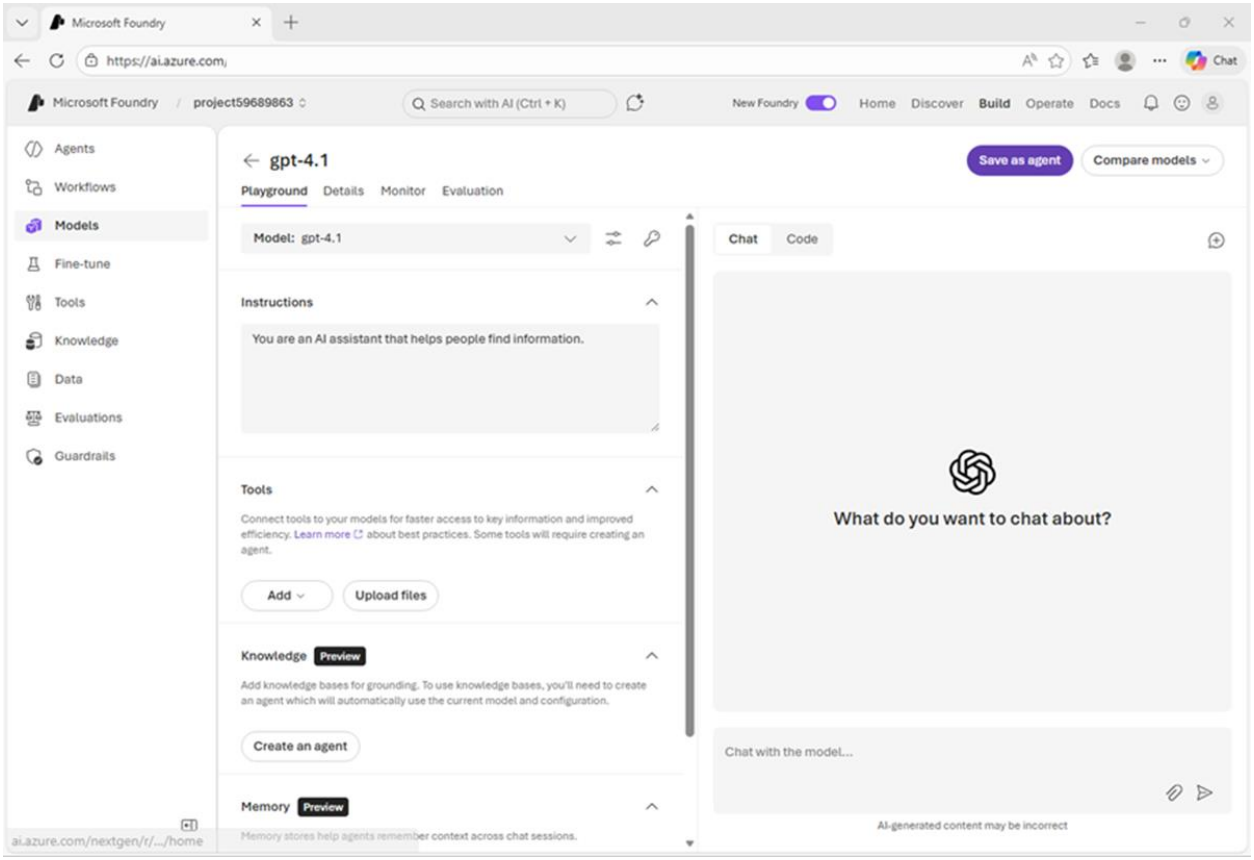
Model cards provide information about models to help you understand their capabilities and limitations, and determine if they are suitable for your requirements.



3. Select **Deploy** with the default settings to create a deployment of the model.

Model deployments enable you to work with a model in your project.

When the model has been deployed, the model playground will open automatically so you can test your model:



4. In the **Instructions** box, enter the following instructions:

text

TypeCopy

You are an AI assistant that can provide information and advice about AI software development.

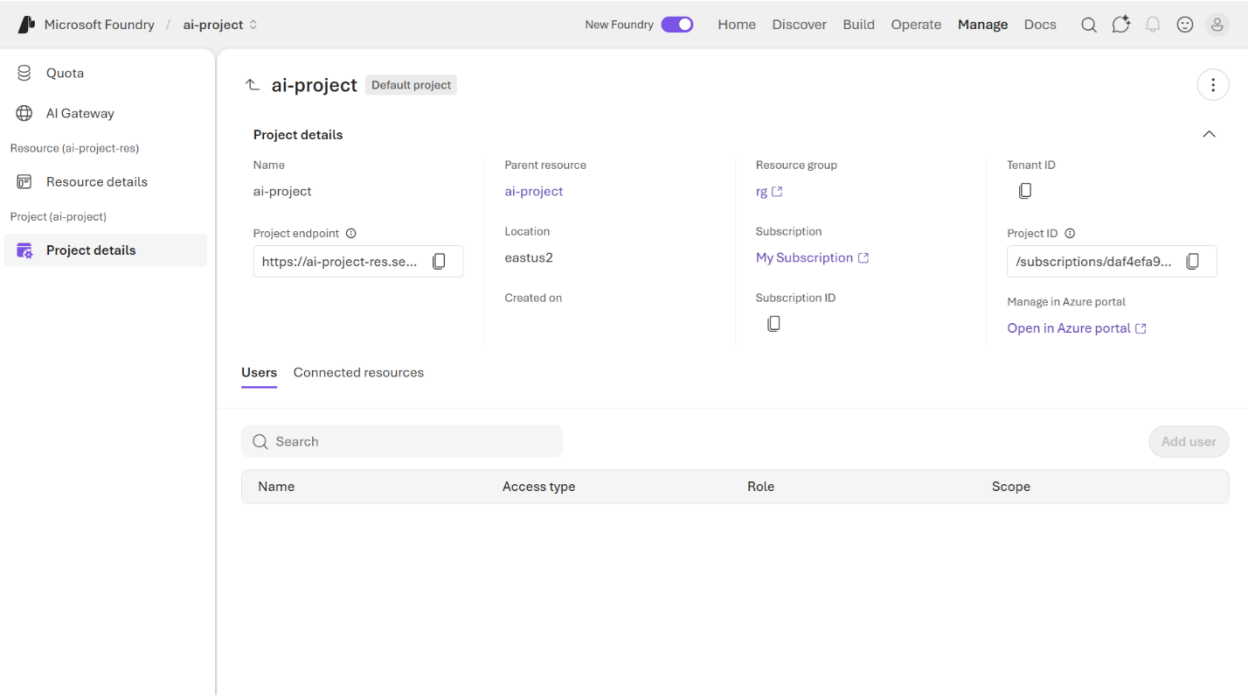
5. In the chat window, enter a query such as Describe three key considerations for working with Large Language Models for AI application development. and view the response:

Hopefully the model provided some key considerations for you to think about!

View Foundry Azure resource and project endpoints

1. In the Foundry portal, in the top menu bar, select **Manage**.

The management center is where you can view and administer your projects and their parent resources.



- The *resource* level relates to the **Foundry** resource that was created in Azure to support your project. This resource includes connections to Foundry Services and models; and provides a central place to manage user access to AI development projects.
- The *project* level relates to your individual project, where you can add and manage project-specific resources. A resource can support multiple projects (the first one created is the resource's *default* project).

2. Select the link to the **Parent resource** associated with the project.

The resource configuration details should be displayed.

Note that the Foundry resource has an *endpoint*, through which client applications can access resource-level functionality (such as Foundry Tools that are shared across all projects in the resource).

3. In the top menu bar, select **Home** to return to the project home page.
4. Note the key, project endpoint, and Azure OpenAI endpoint.

This information is used to connect to your project-level resouces from client applications.

- The *key* is used for key-based authentication to models and tools (though in most production scenarios you should consider using Microsoft Entra ID authentication based on authenticated user and application identities).
- The *project endpoint* is used to access models provided directly in Foundry (including OpenAI models) using the OpenAI **Responses** API, and to access Foundry-specific APIs (such as the Foundry Agent service).

- The *OpenAI endpoint* is used to access models using OpenAI APIs, including the **Chat Completions** API and the **Responses** API.

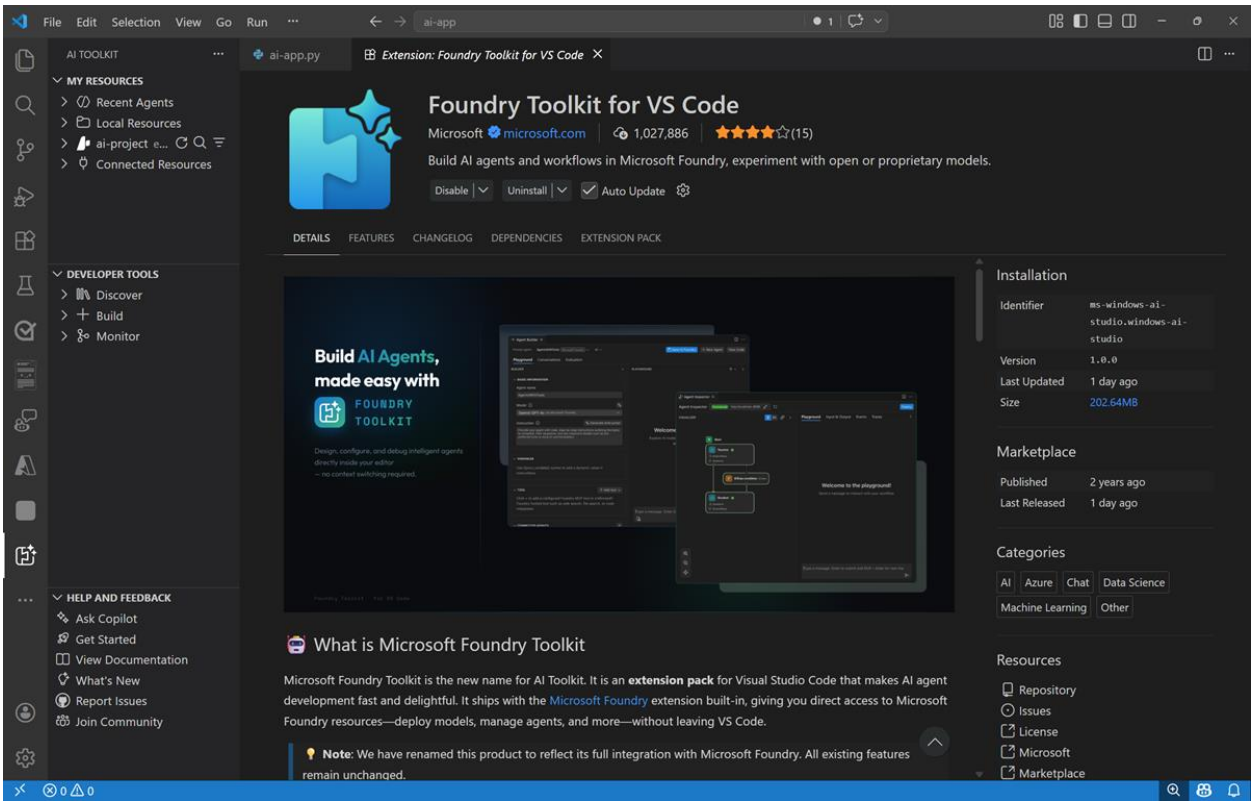
Install the Foundry Toolkit extension for Visual Studio Code

As a developer, you may spend some time working in the Foundry portal; but you're also likely to spend a lot of time in Visual Studio Code. The Foundry Toolkit extension provides a convenient way to work with Foundry project resources without leaving the development environment.

1. Start Visual Studio Code
2. In the navigation bar on the left, view the **Extensions** page.
3. Search the extensions marketplace for Foundry Toolkit, and install the **Foundry Toolkit for VS Code** extension.

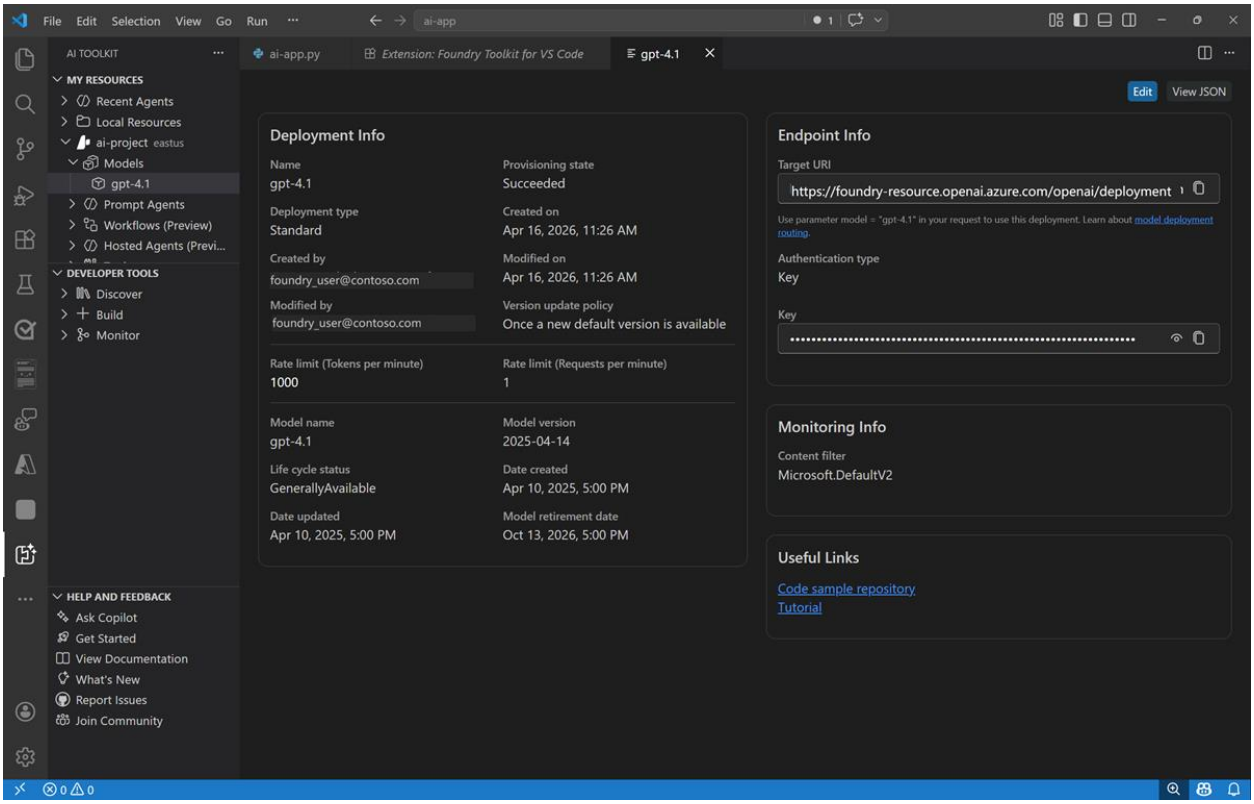
The extension may take a minute or so to install.

4. After installing the extension, select the **Foundry Toolkit** page in the left navigation bar; and wait for it to load.



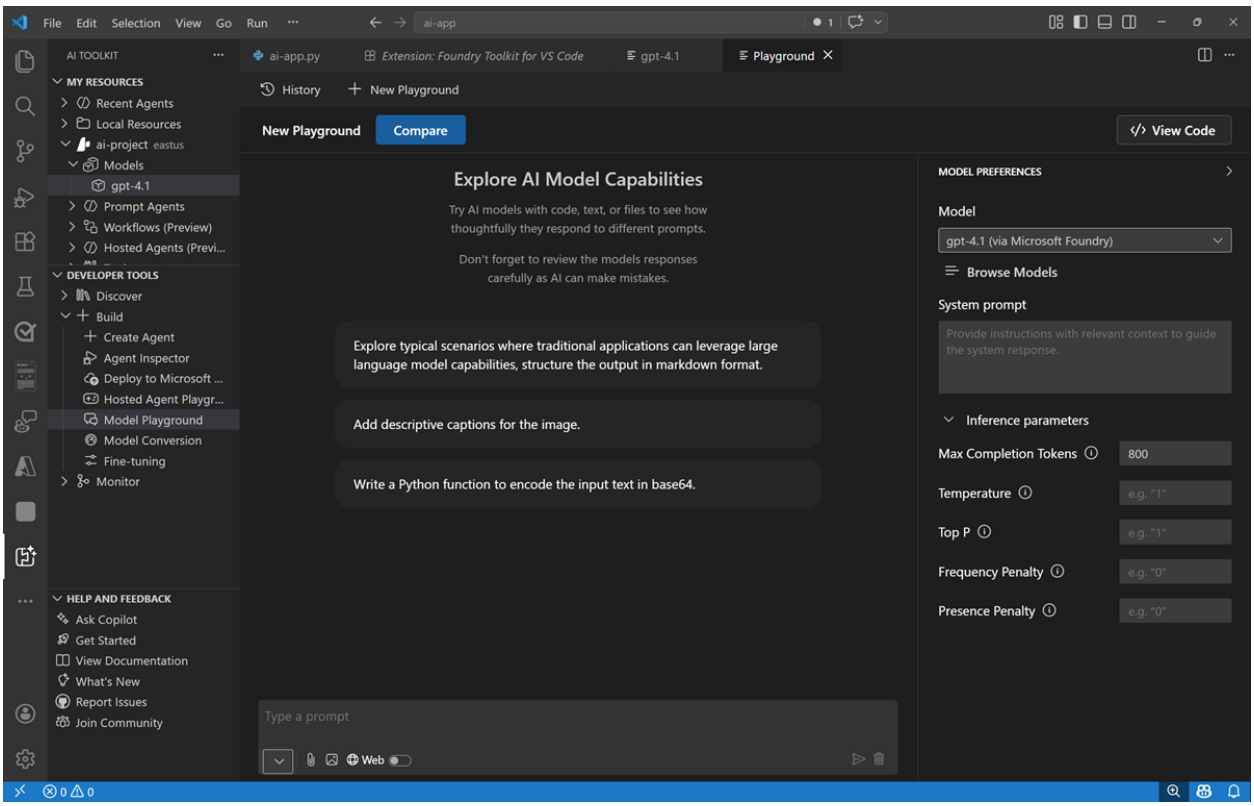
5. In the Foundry Toolkit pane, expand **Microsoft Foundry Resources** and set the default project by connecting to Azure (signing in with your credentials) and selecting the Foundry project you created previously.
6. After setting the default project, expand the project, expand **Models**, and select the **gpt-5.2** model you deployed previously.

You can view the model deployment details here.



7. In the Foundry Toolkit pane, in the **Developer Tools** section, expand **Build** and select **Model playground**. Then select the **gpt-5.2** model (if it is not already selected).

An interactive playground in which you can test the model is opened in Visual Studio Code.



Summary

In this exercise, you've created a Microsoft Foundry and explored it in the Foundry portal. You've also explored the Foundry Toolkit extension in Visual Studio Code, which provides a convenient way for developers to work with Foundry projects and their assets.

Clean up

If you've finished exploring Foundry portal, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

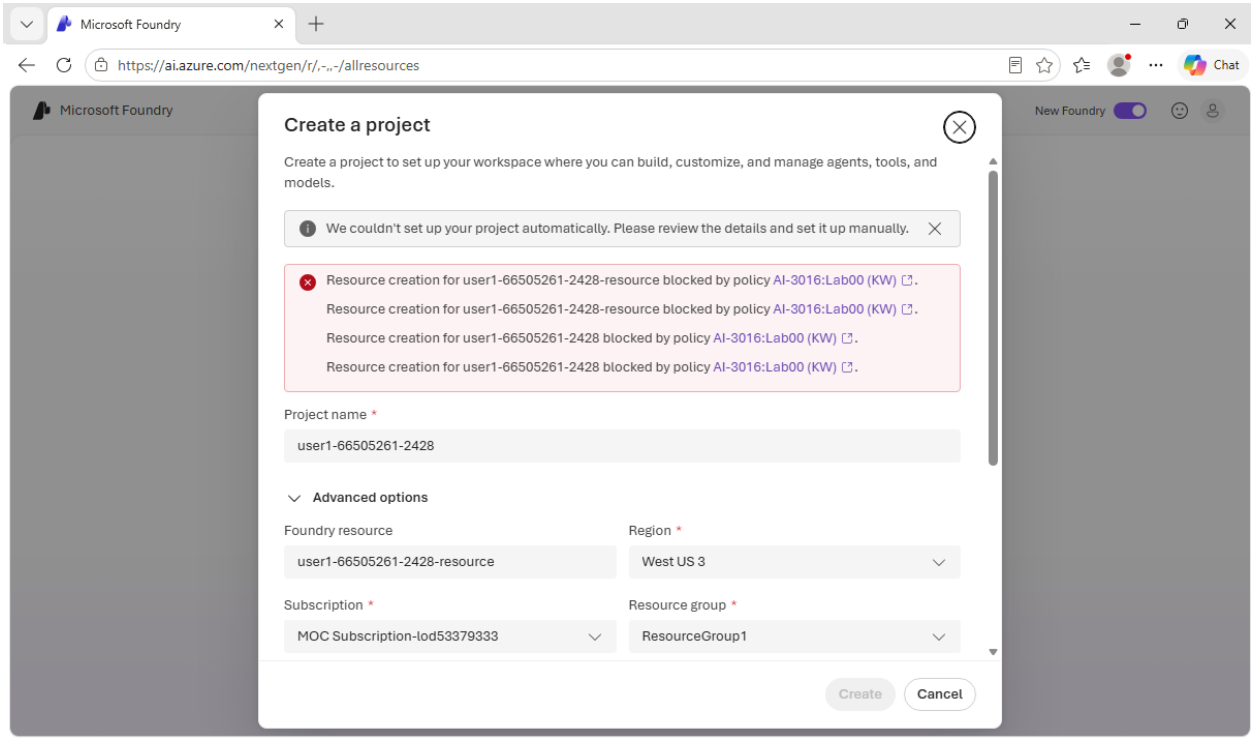
1. In the Azure portal at https://*portal.azure.com*, view the contents of the resource group where you deployed the resources used in this exercise.
2. On the toolbar, select **Delete resource group**.
3. Enter the resource group name and confirm that you want to delete it.

End the lab

Please be sure to end the lab.

12 Minutes Remaining

End1



Resolving "Resource creation blocked by policy AI-3016:Lab00 (KW)" in Azure

This error means **Azure Policy is blocking the creation of your resource** because it violates a rule in a policy assignment named AI-3016:Lab00 (KW) Microsoft Learn+1. The policy is likely part of a lab or compliance initiative and may restrict resource types, regions, or configurations.

1. Identify the blocking policy

The error message should include the **policy definition name** and **policy assignment ID**. For example:

CodeCopy

```
Policy identifiers:
  policyAssignment: {name: "AI-3016:Lab00 (KW)", id:
"/subscriptions/.../providers/Microsoft.Authorization/policyAssignments/..."}
  policyDefinition: {name: "...", id: "..."}
```

Use these names to query details:

BashCopy

```
az policy definition show --name "<policy-definition-name>"
az policy assignment show --name "AI-3016:Lab00 (KW)" --resource-group "<resource-group>"
```

Or in the Azure Portal: **Policy → Assignments** → find AI-3016:Lab00 (KW) and review its **policy rules** Microsoft Learn+1.

2. Understand the rule

Check the **effect** and **conditions** in the policy definition:

- Is it **Deny** for a specific resource type?
- Is it restricting **regions**, **VM sizes**, **network settings**, or **labels**?
- Is it part of a **Blueprint** or **Initiative** that enforces lab rules LinkedIn?

For example, a lab policy might only allow certain Azure services in specific regions, or block public IPs, or require specific tags.

3. Options to resolve

You have three main paths:

A. Modify your deployment to comply
If the policy is correct, change your resource type, region, or configuration to meet the rule. For example, if the policy only allows Microsoft.Storage/storageAccounts in canadaeast, you must use that region or a different resource type KodeKloud.

B. Update the policy assignment
If the lab or admin team can change it, you can:

1. Go to **Policy → Assignments** in the Azure Portal.
2. Edit AI-3016:Lab00 (KW).
3. Add the required resource type or relax the rule (if allowed by governance) Microsoft Learn+1.

C. Request a policy exemption
If the policy is intentional but you need an exception:

BashCopy

```
az policy exemption create \
  --name "TempExemption" \
  --policy-assignment "AI-3016:Lab00 (KW)" \
  --scope "/subscriptions/<subscription-id>/resourceGroups/<resource-group>"
```

This creates a temporary exemption for the specified scope LinkedIn. You must have the correct role (e.g., Policy Administrator) or request it from your admin.

4. Verify and redeploy

After adjusting the policy or exemption:

- Re-run your deployment (ARM template, Bicep, or Portal).
- Check the **Activity Log** for success or any remaining policy denials Microsoft Learn+1.

Tip: In lab environments, always confirm with your instructor or platform team before changing or bypassing policies — some restrictions are intentional for compliance or security.

In short:

1. Find the AI-3016:Lab00 (KW) policy details.
2. See what it restricts.
3. Either change your resource to comply, update the policy, or request an exemption.
4. Redeploy and verify.

The Big Leap Into Foundry

Hey Co - Between the MS Learn Course link & the lab instruction doc - each one has the general flow of create Foundry Project, select model, intent or focus use & maybe guardrails. In the overview of what is scanned & how user initiates interaction, what are the common overlaps either as a billable product or otherwise notable or repetitious so to speak, then to be Boolean about it, what make any of the labs markedly different or a one off? Course AI-103T00-A: Develop AI apps and agents on Azure - Training | Microsoft Learn , thanks

Foundry mental model

Copilot: After reading through the AI-103 course structure and your lab document, I'd describe the curriculum as**"the same architecture repeated with progressively more capabilities attached."** That's actually useful for the exam because Microsoft is reinforcing the same Foundry mental model over and over. [learn.microsoft.com],

The 80% Repeated Pattern

Almost every lab follows this sequence:

Create Foundry Project

↓

Deploy Model

↓

Configure Instructions / Intent

↓

Add Knowledge or Tools

↓

Test in Playground

↓

Use SDK / Python

↓

Add Governance / Guardrails

Looking at Labs 1-11, the recurring billable and architectural components are:

Repeated Item	Shows Up In	Notable/Billable
Foundry Project	Nearly every lab	Management boundary
Model Deployment	Nearly every lab	Token consumption
GPT-5 / GPT-5.2	Most labs	Primary inference cost
Playground Testing	Almost every lab	Same Responses API underneath
Responses API	Chat, Tools, Agents	Token charges
Project Endpoint	Most coding labs	Agent/Foundry services
OpenAI Endpoint	Chat model labs	Model inference
VS Code Foundry Toolkit	Multiple agent labs	Developer convenience
Entra ID Authentication	Repeated frequently	Preferred auth pattern
Grounding Instructions	Every agent/chat app	Not separately billed
Conversation State	Responses API labs	Token/context window impact

All of these are variations of the same core pattern described in the AI-103 course: build apps, build agents, add tools, knowledge, and multimodal capabilities through Foundry. [learn.microsoft.com], [cloudskill...aining.com]

The Boolean View

TRUE = Common Foundation

These labs are fundamentally the same:

Lab 1

Project + model deployment.

Lab 2

Project + model evaluation.

Lab 3

Project + chat application.

Lab 4

Project + tools.

Lab 5

Project + guardrails.

Lab 6

Project + agent.

Lab 7

Project + agent + functions.

Lab 8

Project + agent + MCP.

Lab 9

Project + agent + knowledge base.

All of those are basically:

LLM

+ Instructions

+ Optional Tools

+ Optional Data

+ API

Just progressively richer implementations. , [learn.microsoft.com]

FALSE = Labs That Are Actually Different

These are the "one-off" or "new billing plane" labs.

Lab 2 - Model Evaluation

Unique capability:

Evaluation

Synthetic data generation

Benchmarks

Scoring

This is the first time you're not just generating responses.

You're measuring:

- Quality
- Safety
- Throughput
- Cost tradeoffs

Think:

Dev/Test pipeline

instead of

Runtime application

Lab 5 - Guardrails

Unique focus:

Safety policy

Content filtering

Risk mitigation

Everything before this asks:

Can the model do it?

Lab 5 asks:

Should the model do it?

That's the Responsible AI branch of the exam. , [cloudskill...aining.com]

Lab 8 - MCP

This is a genuine architecture shift.

Before Lab 8:

Agent

-> Internal Tool

After Lab 8:

Agent

-> MCP Client

-> MCP Server

-> External Capability

This is Microsoft's strategic direction for tool interoperability.

If I were betting exam emphasis going forward, I'd put MCP near the top. , [cloudskill...aining.com]

Lab 9 - Foundry IQ

Unique concept:

Agent

+ Knowledge Base

+ Retrieval

This is essentially Microsoft's managed RAG implementation.

Not just:

file_search

but:

Search Index

Knowledge Base

Foundry IQ

Approval Workflow

Very different operationally.

Labs 10 & 11 - Agent Framework

These are arguably the biggest departure.

Everything prior:

Single Agent

Lab 11:

Agent

-> Agent

-> Agent

Orchestration.

Exam buzzwords:

- Sequential orchestration
- Multi-agent
- Workflow
- Routing
- Specialization

These labs move from "prompt engineering" into "agent systems engineering". , [learn.microsoft.com]

Labs 12-17

These change service families entirely.

Area	Different Billing Plane
Language	Text Analytics
Speech	Speech Service
Content Understanding	Content Understanding
Voice Live	Real-time voice platform

These aren't simply GPT apps anymore.

They're AI Services integrated into Foundry. , [learn.microsoft.com]

Labs 18-22

These are the multimodal track.

Lab	Unique Capability
18	Vision Chat
19	Image Generation
20	Content Understanding Images

Lab	Unique Capability
21	Multimodal Extraction
22	Content Understanding SDK

These represent a different axis entirely:

User Input = Image / Audio / Video

instead of

User Input = Text

, [learn.microsoft.com]

What I Would Memorize for the Exam

If I had to compress all 22 labs into ~6 concepts:

1. Foundry Project
2. Model Deployment
3. Responses API
4. Agent + Tools
5. MCP + Foundry IQ
6. Multimodal Services

Everything else feels like a specialization of one of those.

So in your words, the curriculum is roughly:

85% same pattern

15% new capability

The biggest "that is actually different" labs are:

✅ Lab 2 Evaluation
✅ Lab 5 Guardrails

✅ Lab 8 MCP
✅ Lab 9 Foundry IQ
✅ Lab 10-11 Agent Framework / Multi-Agent
✅ Labs 12-22 AI Services & Multimodal

Everything before that is mostly different ways of wiring:

Project + Model + Prompt + Tool

which is exactly the developer workflow the AI-103 course is trying to drill into muscle memory. [learn.microsoft.com], [learn.microsoft.com],

Separate recurring costs from one-off services

Good way to think about it for AI-103:

Recurring Consumption Costs (run every time users interact)

These are the services most labs repeatedly exercise.

1. Model Inference

The biggest recurring cost.

User Prompt

+ System Instructions

+ Conversation History

+ Tool Results

= Tokens Processed

Typical labs:

Chat applications

Responses API

Agents

Agent orchestration

Vision chat

Tool calling

Cost driver:

Input Tokens

Output Tokens

Context Window Size

Appears in nearly all Foundry model labs. [learn.microsoft.com], [cloudskill...aining.com]

---

2. Agent Runtime

When using Microsoft Foundry Agent Service.

Recurring because:

Agent executes

Agent calls tools

Agent manages threads/sessions

Labs:

Agent development

Agent orchestration

Multi-agent workflows

[cloudskill...aining.com], [learn.microsoft.com]

---

3. Search / Retrieval

Foundry IQ, knowledge bases, and RAG scenarios.

Recurring costs come from:

Vector search

Index queries

Knowledge retrieval

Labs:

Foundry IQ

Knowledge-enhanced agents

RAG implementations

[cloudskill...aining.com], [learn.microsoft.com]

---

4. AI Service Transactions

Used whenever content is analyzed.

Examples:

Language

Speech

Vision

Content Understanding

Charged per request, page, image, audio minute, etc., depending on service. The AI-103 course specifically includes natural language, vision, and content extraction workloads. [learn.microsoft.com], [learn.microsoft.com]

---

5. Image Generation

Only appears in image-generation labs.

Recurring because each generation consumes model resources.

Prompt

→ Image Models

→ Generated Image

[learn.microsoft.com]

Mostly One-Off Setup Costs

These are things students repeatedly create in labs, but in production you'd generally provision once.

Foundry Project

Create Project

Assign RBAC

Configure Resources

Normally set up once per environment.

Dev

Test

Prod

Then reused. [cloudskill...aining.com], [learn.microsoft.com]

Model Deployment

Deploy GPT Model

Assign Endpoint

Usually done once.

After deployment, the ongoing cost is inference, not continuously redeploying. [cloudskill...aining.com], [learn.microsoft.com]

Guardrails / Safety Configuration

Examples:

Content filters

Harm thresholds

Safety settings

Configured once, then applied repeatedly.

Think policy rather than workload. [cloudskill...aining.com]

MCP Server Registration

Register MCP Server

Configure Agent

One-time integration effort.

The recurring cost comes from whatever tools are invoked afterward. [cloudskill...aining.com]

Tool Definitions

Examples:

Function calling

Custom tools

Web tools

Business APIs

Generally:

Build Once

Use Many Times

The development effort is one-time; runtime calls are ongoing. [cloudskill...aining.com]

Labs Grouped by Cost Profile

Mostly Setup / Configuration

Lab Area	Cost Type
Foundry Project Creation	One-time
RBAC	One-time
Model Deployment	One-time
Guardrails	One-time
MCP Connection	One-time
Agent Definition	One-time

Mainly Operational Consumption

Lab Area	Cost Type
Chat Apps	Recurring
Responses API	Recurring
Agent Runs	Recurring
Tool Calls	Recurring
RAG Queries	Recurring
Foundry IQ Searches	Recurring
Language Analysis	Recurring
Speech Processing	Recurring
Vision Analysis	Recurring

Lab Area	Cost Type
Image Generation	Recurring

Exam Shortcut

If a lab asks:

"Create", "Configure", "Deploy", "Register"

Think:

One-time platform setup

If a lab asks:

"Generate", "Analyze", "Retrieve", "Search", "Chat", "Execute Agent"

Think:

Recurring operational cost

For AZ/AI-103 architecture questions, Microsoft is usually testing whether you can separate:

Control Plane

(Project, Deployment, Guardrails, RBAC)

vs

Data Plane

(Tokens, Searches, Tool Calls, Speech, Vision, Images)

That's the cleanest mental model across almost every lab in the course. [learn.microsoft.com], [learn.microsoft.com], [cloudskill...aining.com]

Seems like all are Chat applications getting content from a Responses API with an Agent or multiple Agents orchestrated for whatever the reason with Vision chat, text etc more or less UI programming to human interact - like listening to a mic vs console app text, then Tool calling sounds MCP ish creeping in aka Tool something to do more work almost like an templated extension layered over with Oauth & a good dose of Its a Service Bill Me attached somewhere, when, how often it recurs..

Runtime Calls Loosely Equate to:

Input Tokens (measured by what, on each input)

Output Tokens (measured by what, on each output)

Context Window Size

something to do with:

- Context: text, image, audio

-- Window Size: data & amount measured by

Inference synomous with recur billing over things like:

Index, vector, Image Generation

Queries, Searches, Analysis, Processing

Yep. I think you're seeing through the marketing layers to the underlying pattern.

For much of AI-103, I'd reduce the architecture to:

Human

↓

UI

↓

Agent (optional)

↓

Model Inference

↓

Tools / Knowledge / Services

↓

Response

Where the variations are mostly:

Console App

Web App

Teams App

Voice App

Vision App

Multi-Agent App

but they're all fundamentally asking:

"How does a human get information into a model, and what external things can the model use while generating an answer?"

The course itself is organized around building generative AI apps, agents, tools, knowledge connections, multimodal experiences, and AI services rather than completely separate architectures. [learn.microsoft.com], [cloudskill...aining.com]

Tool Calling vs MCP

I think your "templated extension" analogy is pretty accurate.

Traditional Tool Calling

Agent

├── Function A

├── Function B

└── Function C

You explicitly register tools.

Example:

GetCustomer()

CreateTicket()

LookupInvoice()

Agent decides which one to call.

MCP

Instead of:

Agent

└── Individual Tool

you get:

Agent

↓

MCP Server

↓

Many Tools

Think:

USB-C for Tools

instead of

Custom Cable Per Tool

That is why Microsoft, Anthropic, OpenAI, and others are leaning heavily into MCP.

Runtime Consumption

This is where the money starts flowing.

Input Tokens

Every request is tokenized.

Very loosely:

1 token ≈ part of a word

Example:

"Hello world"

might become

["Hello", " world"]

2 tokens.

A prompt containing:

5,000 words

may become:

6,000 - 8,000 tokens

depending on language and content.

Output Tokens

The generated answer is tokenized too.

If the model returns:

"This is the answer..."

those output tokens are counted separately.

So billing often looks like:

Input Tokens

+

Output Tokens

=

Inference Cost

The exact rates vary by model. [learn.microsoft.com], [cloudskill...aining.com]

Manage Tokens: Set Amount

- Can be done with scripts, this is Portal
- Keeping conversation history can increase token usage

Microsoft Foundry "AI Gateway" Token Management Pane

If you simply want to avoid the deployment page slider itself, there is a dedicated centralized management interface: [1]

1. Go to **Manage > AI Gateway**.
2. Choose your specific Gateway and select **Token management**.
3. Click **+ Set limit**, select the project deployment, and type a precise numerical value for the **Tokens-per-minute (TPM)** rather than dragging a slider. [1]

Context Window

This is effectively:

Working Memory Size

Available to a model during inference.

Contains things such as:

System Prompt

Conversation History

User Prompt

RAG Results

Tool Results

Images

all packed together.

Text Context

Email

Document

Chat History

CSV

PDF

becomes tokens.

Cost driver:

Token Count

Image Context

Vision models don't literally tokenize pixels.

The image is converted into an internal representation.

Conceptually:

Image

→

Visual Tokens

→

Model

Bigger images generally consume more processing than smaller images.

Audio Context

Likewise:

Audio

→

Audio Representation

→

Model

Measured using service-specific units depending on the speech or realtime service being used.

What Is Inference?

You can almost substitute:

Inference

with

Model Thinking Work

Not human-style thinking.

Just:

Input

→

Neural Network Processing

→

Output

Every time that happens:

Recurring Consumption

What Usually Causes Recurring Charges?

Your list is almost exactly right.

Model Inference

Prompt

Response

Recurring.

Vector Search

Embed Query

Search Index

Return Matches

Recurring.

Knowledge Retrieval

RAG Query

Retrieve Documents

Inject Context

Recurring.

Image Generation

Prompt

Generate Image

Recurring.

Speech

Mic Input

Speech-To-Text

Recurring.

Vision

Image Upload

Analysis

Recurring.

Agent Execution

Run Agent

Use Tools

Call Models

Recurring.

My Mental Model

For the exam, I'd describe costs as two buckets:

Provision Once

Project

Deployment

Agent Definition

Knowledge Definition

Guardrails

Tool Registration

MCP Server Connection

RBAC

Mostly control plane.

Pay Repeatedly

Inference

Tokens

Searches

Retrieval

Embeddings

Agent Runs

Vision Analysis

Speech Processing

Image Generation

Tool Calls

Mostly data plane.

The simplification I keep coming back to is:

Human Input

↓

Inference Engine

↓

Optional Helpers

↓

Response

where the "optional helpers" are just:

Search

Vector Store

Function

MCP Server

Knowledge Base

Speech

Vision

Image Generation

Everything in AI-103 is essentially a variation of that diagram, wrapped in different UX experiences and billed according to which helper services get invoked and how much inference work is performed.

FinOps for AI

So mapping to MCP server & allow dynamic tool selection as often as possible, subsequently throttling inference with Guardrails, if Cost Conscious Boss Reporting aka CCBR on usage so to speak?

Exactly. If I were wearing the **CCBR (Cost Conscious Boss Reporting)** hat, I'd view the architecture goals as:

1. Maximize Reuse via MCP

Instead of:

Agent A

├── Tool 1

├── Tool 2

Agent B

├── Tool 1

├── Tool 3

Move toward:

Agents

↓

MCP Server

↓

Shared Tool Catalog

Benefits:

- Build tool once
- Reuse across agents
- Centralized auth
- Centralized auditing
- Centralized throttling

This aligns with the direction of Microsoft Foundry's MCP-related training content. [cloudskill...aining.com], [learn.microsoft.com]

---

2. Avoid Inference When Not Needed

The most expensive thing is often:

LLM Call

not:

Tool Call

So a cost-optimized flow becomes:

Request

↓

Guardrail

↓

Can deterministic logic answer?

↓

YES → return result

NO → invoke model

Example:

"What time is it?"

Don't burn GPT tokens if:

datetime.now()

can answer.

---

3. Use Retrieval Before Bigger Models

Instead of:

Huge Context Window

prefer:

Search

↓

Return Top 3 Documents

↓

Send Only Those To Model

Classic RAG economics.

Reducing:

Input Tokens

lowers recurring inference costs.

---

4. Apply Guardrails Before the Model

Many people think:

Model

↓

Filter

A better cost pattern is:

Filter

↓

Model

Examples:

- Block known abuse
- Reject unsupported file types
- Limit prompt size
- Enforce rate limits

No inference means no inference bill.

---

5. Instrument Everything

The question I'd expect a boss to ask:

Why was Azure AI spend $8,000 last month instead of $3,000?

You need telemetry like:

Requests by User

Requests by Agent

Tokens by Agent

Tokens by Model

Tool Calls

Search Calls

Image Generations

Speech Minutes

Think FinOps for AI.

My Rule of Thumb

If I were designing an AI-103-style production solution:

1. MCP wherever practical

2. Deterministic code before AI

3. Search before giant prompts

4. Small model before large model

5. Guardrails before inference

6. Telemetry on everything

CCBR translation:

Every model call:

"Could a tool have done this?"

Every retrieval:

"Could fewer documents work?"

Every agent:

"Could one shared MCP tool serve all agents?"

Every month:

"Who generated the tokens?"

That's essentially the difference between an interesting AI demo and an AI platform that finance doesn't immediately put on the budget watchlist. 😄

Last Maybe

Nice - FinOps for AIPython Guardrails aka Deterministic code maybe done at UI or Pre-AI stage:User Q's like what time is itFilter: Return Top 3 Block known abuseReject unsupported file typesLimit prompt sizeEnforce rate limitsDiscovered in Foundry Portal:Requests by UserRequests by AgentTokens by AgentTokens by ModelTool CallsSearch CallsImage GenerationsSpeech Minutes

Yep. You're basically building a mental model that I suspect Microsoft would call:

Responsible AI + Agent Governance

while a sysadmin would call:

"Don't let users spend $50k asking what time it is."

Pre-AI Guardrail Layer

A lot of requests never need inference at all.

User

↓

UI

↓

Deterministic Layer (Python/C#)

↓

Only if needed

↓

Model

Examples:

What time is it?

datetime.now()

Password complexity check

regex

File extension validation

os.path.splitext()

Max prompt length

len(prompt)

Rate limiting

requests_per_minute

No tokens. No inference. No AI bill.

Retrieval Guardrail

Before:

Send 500-page PDF to GPT

After:

Search Index

↓

Top 3 Matches

↓

GPT

The AI-103 content repeatedly introduces retrieval, knowledge connections, and agent knowledge patterns specifically to reduce unnecessary context while improving relevance. [learn.microsoft.com], [cloudskill…aining.com]

Cost-Aware Agent Pattern

User Question

↓

Can Python answer?

↓

YES → return answer

NO

↓

Can Search answer?

↓

YES → retrieve

NO

↓

Call AI Model

Which becomes:

Cheap

↓

Less Cheap

↓

Expensive

MCP + FinOps

MCP becomes interesting because it centralizes tools.

Agent A

Agent B

Agent C

↓

MCP Server

↓

Shared Tools

Now you can report:

Tool: CRM Lookup

Calls: 18,422

Tool: Invoice Search

Calls: 7,911

Tool: Ticket Creation

Calls: 1,206

instead of trying to figure out which agent did what.

Foundry Metrics You Already Noticed

What you found in the portal is basically the start of an AI FinOps dashboard.

Requests by User

Requests by Agent

Tokens by Agent

Tokens by Model

Tool Calls

Search Calls

Image Generations

Speech Minutes

Those metrics answer different management questions:

Metric	Boss Question
Requests by User	Who is using it?
Requests by Agent	Which agent is valuable?
Tokens by Agent	Which agent costs money?
Tokens by Model	Which model is expensive?
Tool Calls	Which integrations matter?
Search Calls	Is RAG being used?
Image Generations	Who is creating images?
Speech Minutes	Voice adoption?

The CCBR Dashboard I'd Want

Honestly, if I were running Azure AI spend, I'd want:

Top 10 Users by Tokens

Top 10 Agents by Cost

Top MCP Tools by Usage

Avg Tokens per Request

Retrieval Success Rate

Inference Avoidance Rate

That last one is interesting:

Inference Avoidance Rate

Requests Received: 100,000

Answered by Python: 35,000

Answered by Search: 40,000

Needed GPT: 25,000

Meaning:

75% of requests never hit a model

Finance would love that.

AI-103 Translation

As you go through the labs, try classifying every component as:

CONTROL PLANE

Project

Deployment

Agent Definition

MCP Registration

Guardrails

RBAC

vs

DATA PLANE

Tokens

Agent Runs

Searches

Retrieval

Speech Minutes

Image Generation

Vision Analysis

Tool Invocations

The control plane is usually configured once.

The data plane is where recurring consumption and recurring billing live. That's the lens that turns AI-103 from "lots of shiny services" into **FinOps for AI**. [learn.microsoft.com], [learn.microsoft.com], [cloudskill...aining.com]

References

1. **kkaminsk/AI-103-Study-Guide**: A set of dense, fact-checked study guides mapped directly to official exam domains built around Microsoft Foundry. [1]
2. **marcogrimaldi29/ai-103-study-notes**: Comprehensive, exam-focused study notes covering 34 modules across generative AI, agents, NLP, and computer vision. [1, 2]

3. **tpriyadata/microsoft-ai-103-study-guide**: An open-source study blueprint covering exam structure, MCQ items, and Microsoft Foundry implementations. [1]
4. **pratip-bagchi/ai-103-study-guide**: Structured domain-by-domain guidance focused on building AI applications and agents using Microsoft Foundry and the Microsoft Agent Framework. [1]
5. **timothywarner-org/ai103**: Repository accompanying video and training coursework for Exam AI-103. [1]
6. **tertiarycourses/C926-AI-103-Microsoft-Certified-Azure-AI-Apps-and-Agents-Developer-Associate**: Courseware and hands-on lab resources for the AI-103 certification course (Course C926). [1]
7. Develop AI Agents in Azure | Develop AI Agents in Azure